

Ethical Hacking — Exam Study Guide

Part I · Casing the Establishment (Footprinting · Scanning · Enumeration)

Purpose & method. This is a reorganised, study-focused rewrite of the course theory. It preserves the original **narrative flow** (essential for how you study) and **every non-trivial technical detail** — commands, flags, port numbers, worked examples — while trimming pure trivia (tutorial authors, publication dates, filler). Each technique is framed around the **seven dimensions of depth the course demands**: *definition/description · technical mechanism · preconditions/assumptions · information collected & impact · tools/commands · detection & countermeasures · limitations*. Detail from the Anki deck has been folded into the relevant sections wherever it added precision the theory text lacked.

Table of Contents

- **Chapter 1 — Footprinting**
 - 1.0 Where footprinting sits · 1.1 Anonymity (IAAAS, Tor, tor-resolve, socat) · 1.2 Definition & methodology · 1.3–1.4 Scope & Authorization · 1.5 OSINT · 1.6 WHOIS & DNS enumeration · 1.7 DNS interrogation · 1.8 Network reconnaissance
- **Chapter 2 — Scanning**
 - 2.1 Host discovery (ARP/ICMP/Nmap/TCP-UDP) · 2.2 Ping-sweep countermeasures (Loki) · 2.3 TCP/IP fundamentals · 2.4 Port scanning (SYN/Connect/Xmas/FIN/NULL/ACK/Window/UDP) · 2.5 OS detection (active/passive) · 2.6 Storing scan data (Metasploit)
- **Chapter 3 — Enumeration**
 - 3.1 Scanning vs Enumeration · 3.2 Service fingerprinting & vuln scanners · 3.3 Banner grabbing · 3.4 Common services (FTP/Telnet/SMTP/DNS/TFTP/Finger/HTTP/MSRPC/NetBIOS) · 3.5 Advanced (SNMP/BGP/LDAP) · 3.6 Summary
- **Chapter 4 — Hacking Windows**

- 4.1 Why Windows is vulnerable · 4.2 Attack categories · 4.3 Remote password guessing · 4.4 Password-exchange eavesdropping (Kerberos pre-auth) · 4.5 MITM & SMB relay · 4.6 Pass-the-Hash / Pass-the-Ticket · 4.7 Remote unauthenticated exploits · 4.8 End-user app exploits · 4.9 Device-driver exploits · 4.10 Privilege escalation · 4.11 Extracting & cracking passwords (SAM/salting) · 4.12 Cached passwords (LSA Secrets/MSCASH) · 4.13 Backdoors · 4.14 Port redirection · 4.15 Covering tracks (ADS/rootkits) · 4.16 Incident response · 4.17 Defense architecture (WRP/MIC-UAC/BitLocker)
- **Chapter 6 — Cybercrime & APT** (*Ch. 5 Unix Hacking not in source*)
 - 6.1 What defines an APT · 6.2 APT vs non-APT · 6.3 Crime vs espionage · 6.4 Intrusion & evasion · 6.5 The 6 phases · 6.6 Detection/forensics overview · 6.7 Historical campaigns (Aurora/Anonymous/RBN) · 6.8 GhostNet case study · 6.9 IoCs & persistence · 6.10 Order of Volatility (RFC 3227) · 6.11 Memory forensics (Volatility) · 6.12 File/process capture workflow · 6.13 Linux APT scenario · 6.14 Malware/botnets (Poison Ivy/TDSS) · 6.15 The 14-step anatomy · 6.16 Detection & mitigation
- **Chapter 7 — Remote Connectivity & VoIP Hacking**
 - 7.1 Dial-up hacking (wardialing, penetration domains, 12-step framework) · 7.2 PBX & voicemail (default creds) · 7.3 VPN hacking (PCF/Google dorks, IPsec probing, IKE Aggressive mode, Citrix App Breakout) · 7.4 VoIP (SIP scanning/TFTP pillaging, enumeration, RTP interception, SIP-INVITE DoS) · 7.5 Remote-access security tips
- **Chapter 8 — Wireless Hacking**
 - 8.1 Aircrack-ng & WEP case study · 8.2 802.11 fundamentals · 8.3 Session establishment · 8.4 Flawed defenses (hidden SSID/MAC filter) · 8.5 WEP/WPA/WPA2-3 encryption · 8.6 Personal vs Enterprise · 8.7 Four-way handshake · 8.8 Equipment & antennas · 8.9 Wardriving · 8.10 Sniffing & deauth DoS · 8.11 MAC spoofing · 8.12 WEP cracking (IV/FMS/PTW/ARP replay) · 8.13 WPA-PSK (handshake capture/offline crack) · 8.14 WPA-Enterprise/EAP (LEAP, PEAP/TTLS, AP impersonation) · 8.15 Defense summary
- **Chapter 9 — Hacking Hardware**
 - 9.1 Physical access (lock bumping) · 9.2 Cloning access cards (magstripe, RFID/MiFare) · 9.3 Locked hard disks (ATA Security, hot-swap bypass) · 9.4 U3 USB flash drives (AutoRun/PocketKnife) · 9.5 Default

configurations (routers, ATM, Bluetooth) · 9.6 Reverse engineering (bus sniffing, FCC-ID, firmware reversing & Cramfs mounting)

- **Chapter 11 — Mobile Hacking (Android & iOS)** (*Ch. 10 Web Hacking not in source*)
 - 11.1 Android architecture · 11.2 Android security mechanisms · 11.3 The 4 user defenses · 11.4 Dev/debug tools (ADB) · 11.5 Rooting (Magisk, cross-compiler, SafetyNet) · 11.6 Trojan apps & repackaging (apktool/Smali) · 11.7 Attacking Android (WebKit/RATC, zero-permission, capability leaks, banking trojans, Carrier IQ) · 11.8 iOS history · 11.9 iOS security · 11.10 Jailbreaking (boot-based/remote) · 11.11 Hacking iPhones (FOCUS 11, iKee) · 11.12 Mobile defense summary
-

CHAPTER 1 — Footprinting

1.0 Where footprinting sits: "Casing the Establishment"

This guide follows the theoretical structure of *Hacking Exposed 7: Network Security Secrets & Solutions*, which organises offensive network security into four parts. The first, "**Casing the Establishment**," contains the reconnaissance triad: **footprinting** → **scanning** → **enumeration**. The remaining parts (Endpoint & Server Hacking; Infrastructure Hacking; Application & Data Hacking) build on top of the intelligence gathered here. Footprinting is therefore the *very first* methodical process an attacker runs, and its guiding philosophy is Sun Tzu's *The Art of War*: "**Know yourself and your enemy.**"

1.1 The Hacker's Methodology and Operational Anonymity (IAAAS)

Before any reconnaissance, the attacker secures **anonymity**. The governing principle is the acronym **IAAAS** — "**It's All About Anonymity, Stupid.**" This is the preparatory phase in which the hacker masks their real IP and traffic so that the target's firewalls and logs never record their true origin, rendering the Blue Team's tracing and response ineffective.

The anonymity toolchain (end-to-end workflow)

The tools are chained in a specific order:

1. **Tor** provides the anonymous transport (layered cryptography over a SOCKS proxy).
2. A Tor GUI controller, **Vidalia**, runs alongside **Privoxy**, a web-filtering proxy.
3. Once anonymous, the attacker uses **Google** in the browser to identify high-value ("juicy") targets.
4. To resolve target hostnames to IPs **without leaking DNS**, the attacker uses **tor-resolve** instead of the standard `host` command.
5. **proxychains** forces terminal tools (e.g. Nmap) strictly through the Tor network.
6. **Nmap** maps open services on the targets.
7. **socat** maintains persistent relay connections; **nc** (**netcat**) sends crafted requests to verify server versions.
8. The end goal of this reconnaissance is to find and exploit a vulnerability — to "**pwn**" (own/compromise) the system.

Tor (The Onion Router) — mechanism

Tor builds a **circuitous, randomised route through multiple relay nodes** to obscure the origin:

- The user's client (Alice) contacts a **directory server** to obtain the list of active Tor nodes.
- The client then **autonomously picks a random path** through those nodes to the destination server (Bob).
- Links **between** Tor nodes are strictly encrypted (drawn as **green links** in diagrams). Only the final hop — from the **exit node** to the destination — travels **in the clear** (drawn as a **red link**).

Countermeasure. Organisations can block inbound connections at the perimeter firewall from the **known, catalogued IPs of Tor exit nodes**.

Limitations. Tor introduces **high latency** from the multiple hops, and it natively supports **only TCP**. Protocols such as **UDP or ICMP (a standard ping sweep)** typically fail, which is precisely why the attacker must fall back on tunnelling tools like **socat**.

Vidalia and Privoxy

Vidalia is a (now discontinued) cross-platform GUI for *controlling* Tor. Its control panel exposes: start Tor, stop Tor, set up relaying, view the network, use a new identity, and view a bandwidth graph / message log — letting the user confirm they are "Connected to the Tor network!"

Privoxy is a free **web proxy for privacy** that acts *before* the page is rendered by the browser. It **manipulates cookies, modifies web-page data, and alters HTTP headers**; it is highly effective at filtering unwanted pages and stripping advertisements, and it is heavily customisable.

tor-resolve — leak-free name resolution

Standard DNS resolution over Tor can leak the user's real IP. **tor-resolve** is a small Linux CLI script that **resolves a hostname to an IP directly via Tor**: it connects to a SOCKS proxy that understands the **SOCKS RESOLVE** command, hands over the hostname, and securely returns the IP.

- **Default target:** the local Tor server at **127.0.0.1**, **SOCKS port 9050** (an explicit SOCKS host/port can be supplied).
- **Options:** **-v** verbose; **-x** reverse lookup (PTR record for an IPv4 address); **-5** SOCKS5 (default); **-4** uses the older **SOCKS4a** protocol — **but SOCKS4a does not support reverse DNS lookups**.
- **Example:** **# tor-resolve www.example.com** → returns e.g. **10.10.10.100**.

Nmap — target and port selection (used throughout recon)

Target selection:

- Single IP: **nmap 192.168.1.1**
- Hostname: **nmap www.testhostname.com**
- Range: **nmap 192.168.1.1-20**
- Whole subnet (CIDR): **nmap 192.168.1.0/24**
- Target list from file: **nmap -iL list-of-ips.txt**
- *Default behaviour:* host discovery + scan of the **1000 most common TCP ports**.

Port selection:

- Single port: `nmap -p 22 192.168.1.1`
- Range: `nmap -p 1-100 192.168.1.1`
- Fast scan (100 most common ports): `nmap -F 192.168.1.1`
- All 65,535 ports: `nmap -p- 192.168.1.1`

socat (relay) and **nc** (version checks)

`socat` relays connections through Tor. A practical use is opening a local proxy on `localhost:8080` that forcibly forwards everything through Tor to a target:

```
# socat TCP4-LISTEN:8080,fork
SOCKS4a:127.0.0.1:10.10.10.100:80,socksport=9050 &
```

`nc` (netcat) is used to send specific requests to a server to **verify its version**.

1.2 Footprinting: definition, targets and methodology

Footprinting is the meticulous creation of a profile of the target organisation — seeing exactly what the attacker sees "from the outside looking in." The OSINT-driven steps are essentially **passive** (data is gathered from public sources rather than by hammering the target's own servers), which keeps the attacker below the IDS's radar; note, however, that the later DNS-interrogation and network-reconnaissance steps *do* interact with target infrastructure.

What gets profiled across the organisation's four faces:

- **Internet:** domain names, network blocks, subnets, specific IPs, active TCP/UDP services, system architecture (CPU arch), access-control mechanisms, IDS, system enumeration, DNS hostnames.
- **Intranet:** internal network protocols, internal domain names, internal blocks/IPs, internal TCP/UDP services, architectures, internal access controls, IDS config, internal enumeration.
- **Remote access:** phone numbers, remote system types, authentication mechanisms, VPN setups.
- **Extranet:** domain names, connection sources/destinations, connection types, associated access controls.

The six-step Internet-footprinting methodology:

1. Determine the **scope** of your activities.
 2. Get proper **authorization**.
 3. Gather **publicly available information** (OSINT).
 4. Perform **WHOIS** and **DNS enumeration**.
 5. Conduct **DNS interrogation**.
 6. Execute **network reconnaissance**.
-

1.3 Step 1 — Determine the Scope

Before any technical work, define the engagement boundaries: does footprinting cover the **entire organisation** or only specific **subsidiaries**? It is strongly recommended to identify **all associated entities** at this stage so none becomes an unsecured blind spot.

1.4 Step 2 — Get Proper Authorization

Technical skill without authorization is **legally dangerous**. This is jokingly called navigating "**Layers 8 and 9**" of the **OSI model** — **politics and funding**. Securing **explicit, written authorization** is the ethical hacker's "**Get-out-of-jail-free card**," protecting them from legal repercussions during reconnaissance. (*Note: Steps 1 and 2 are the mandatory legal preconditions that must be completed before OSINT begins.*)

1.5 Step 3 — Gathering Publicly Available Information (OSINT)

The volume of data obtainable **without ever touching the target's internal network** is the heart of footprinting.

Company web pages

The first stop is the target's public site. Attackers frequently find **poorly obscured security configurations** or mistakenly uploaded **asset-inventory spreadsheets**. A key technique is **reviewing the HTML source code**, made far faster by **mirroring the site offline** with tools like **Wget** (Linux) or **Teleport Pro** (Windows). In the offline source, attackers hunt for material buried

in **HTML comment tags** (`<!-- -->`), where developers leave notes about backend systems, passwords, or internal IP structures.

Attackers also **recursively enumerate hidden files and directories** with automated tools like **OWASP DirBuster**. Because directory brute-forcing is **noisy and easily detected by firewalls**, the traffic is typically **proxied** (e.g. **through Privoxy**) to mask its origin. Reconnaissance further looks for **remote-access gateways** to internal resources via the browser (e.g. portals proxying to internal **Microsoft Exchange** servers) and expands to **non-primary sites** (www1, www2, web, test environments, dedicated VPN sites).

Related organisations and physical location

A company rarely exists in a vacuum. Attackers look for **partners and related organisations** that may be **less security-minded** — e.g. a **third-party web-development firm** can provide a backdoor into the main company or a social-engineering avenue.

Physical location enables **dumpster-diving, surveillance, social engineering, and physical intrusion**. Attackers conduct virtual surveillance with **images, Google Earth, and Google Maps Street View**. A particularly devastating technique is **harvesting Wi-Fi MAC addresses from Street View, Google Locations, and Skyhook** to map digital infrastructure to precise physical locations — famously demonstrated in the "**How I Met Your Girlfriend**" presentation at **BlackHat 2010**.

Shodan

Shodan — "the **Search Engine for the Internet of Everything**" — is the first search engine built for **Internet-connected devices rather than websites**. It looks "Beyond the Web" to expose infrastructure ranging from power plants and mobile phones to refrigerators and Minecraft servers. It functions as an **Internet-intelligence tool**: attackers (and defenders) monitor which **physical devices/banners** (routers, webcams, SCADA) are directly reachable on public IPs and how technologies are deployed globally over time.

Harvesting employee information

Employees are usually the **weakest link**. Finding names quickly yields the **corporate e-mail format and system usernames**. Phone numbers link to physical addresses or fuel social engineering, via directories like

Phonenumber.com, 411.com, yellowpages.com. Deeper personal digging (home phones, addresses, SSNs, credit history, criminal records) uses **Blackbookonline.info, peoplesearch.com.**

Social, professional, career, ancestry and photo sites provide psychological ammunition: **Facebook, Reunion.com, Classmates.com, Twitter, LinkedIn, Plaxo, Monster, Careerbuilder, Dice, Ancestry.com, Flickr, Photobucket.** Attackers also exploit **B2B business directories** normally used by sales teams — **JigSaw.com, FullContact, Insideview** — which are often paid services that reward users with points for adding/updating contacts, producing highly accurate databases. To manage the collection, the **OSINT framework (osintframework.com)** is used.

Job postings and resumes

Job postings and resumes are **goldmines for technical footprinting:**

- A posting seeking experience with **"Checkpoint firewalls and Snort IDS"** tells the attacker **exactly which defenses** they face.
- Attackers run targeted Google searches (e.g. company name + **"resume firewall"**) to find current/past employees' resumes, or search **monster.com / careerbuilder.com** directly.
- Special attention goes to **disgruntled and ex-employees**, who may seek revenge (insider-threat potential).
- An employee's **home computer** is a prime target: installing a **keystroke logger** on a poorly secured home PC lets the attacker **steal VPN credentials** and get a **"free ride"** into the target network, impersonating a trusted user.

Monitoring current events and financial data

Corporate instability breeds vulnerability. Attackers watch for **mergers, acquisitions, scandals, layoffs, rapid hiring, reorganisation, outsourcing, and temporary contractors:**

- A **merger/acquisition** blends two networks, which often means security controls are **temporarily reduced or disabled.**
- During **layoffs/scandals**, morale drops, employees update resumes (**leaking internal data**), and unauthorised guests slip by unnoticed.

To track corporate movements, attackers read **SEC filings — the 10-Q (quarterly) and 10-K (annual)** reports — via the **EDGAR database (sec.gov/edgar/search)**, searchable by keyword, company, ticker, CIK, and date range. EDGAR yields authoritative **org charts and structural data**. Attackers complement this with stock/business sites such as **Yahoo! Finance message boards** to pick up internal rumours and sentiment.

Privacy policies and archived information

Paradoxically, a company's **privacy/security policy** can reveal the **exact security mechanisms in place**. And when a company removes leaked data from its live site, attackers simply pivot to **archives — where old copies are often more valuable than current ones**:

- **Archive.org's Wayback Machine** (over **866 billion** archived pages) lets attackers resurrect **deleted configuration files, org charts, and old network diagrams**.
- **Google cache** serves the same purpose for recently changed/removed pages.

Search-engine hacking and data relationships

Standard engines (Google, Bing, Yahoo, Dogpile, Ask) become hacking tools via specialised search strings catalogued in the **Google Hacking Database (GHDB)**. This is **highly passive** — the attacker **queries Google's index/cache, not the target's server**. Key operators: **intitle:** (page title), **inurl:** (URL), **filetype:** / **ext:** (file extension), **site:** (restrict to a domain).

- **Preconditions:** the target admin must have **misconfigured something** — e.g. a bad/ignored **robots.txt** or **Directory Browsing ("Index of")** left enabled on Apache/IIS.
- **What it exposes:** backup DB files (**.sql** , **.bak**), config files with cleartext passwords (**web.config** , **.htpasswd**), exposed login portals, and error messages leaking SQL query structure.
- **Example dorks:** **intitle:"Index of" ".htpasswd" htpasswd.bak** and **inurl:backup intitle:index.of inurl:admin** expose backup directories and password files; the classic **allinurl:tsweb/default.htm** instantly finds Windows servers with **Remote Desktop Web Connection** exposed to the internet.

- **Automation tools: SiteDigger** (foundstone), **Athena**, **Wikto**, **SearchDiggity** — they run a domain against hundreds of preset GHDB queries. SiteDigger buckets its findings (Backup Files, Configuration Manager, Error Messages, Reported Vulnerabilities).
- **Metadata & relationship mapping: FOCA** analyses metadata hidden in published files (PDFs, Word docs) to leak **internal usernames and software versions**; **Maltego** mines and **visually links** people, e-mails, domains and infrastructure into a relationship graph.

OSINT countermeasures

Refer to the **Site Security Handbook (RFC 2196)** and enforce strict policies to **periodically review and completely remove sensitive-but-public data** from the internet. Practically: **scrub metadata** from published documents (e.g. remove the "Admin" author from PDFs/Word), request **de-indexing** via Google Search Console, disable directory browsing, and train staff against **over-sharing** technical details (including photos of badges).

Limitations of OSINT. It produces **enormous noise** — data may be **outdated, invalid, or deliberately false** (disinformation, corporate honeypots).

1.6 Step 4 — WHOIS and DNS Enumeration

With OSINT gathered, the attacker systematically identifies the **technical footprint** (domains, IPs, ports). Global allocation is centrally managed by ICANN, and the data is stored hierarchically across global **WHOIS and DNS servers**.

The "Three R's"

- **Registry** — oversees the **top-level domain** (e.g. **.com** , **.edu**).
- **Registrar** — the **retail company** that sold the domain.
- **Registrant** — the **actual owner** of the domain.

Domain trace (example, **keyhole.com):** start at the top of the hierarchy with **whois.iana.org** → identify the registry/registrar for **.com** (e.g. **verisign-grs.com**) → trace the specific registrar for **keyhole.com** (e.g. **markmonitor.com**) → reach the **Registrant details** (names, phone numbers, admin e-mails) — invaluable for later **social engineering or spoofing**.

IP-address lookups

The same hierarchy applies to IPs. ICANN/IANA is **not** the direct authoritative registry for IPs; instead each **Regional Internet Registry (RIR)** knows the ranges it manages. You can start at **any RIR** — query the wrong one and it will **redirect you to the correct RIR**. Example: looking up `61.0.0.2` starts at **ARIN (`arin.net`)**; if the IP is outside North America, ARIN points to **APNIC (`apnic.net`)**, which reveals the block belongs to the **National Backbone of India**.

Caveat: the source IP found may be **spoofed or masqueraded** — never assume it is the true origin.

Internet infrastructure (needed to understand IP/DNS enumeration)

- **Tier 1** networks form the **global backbone**.
- **Tier 2** networks (large regional ISPs with their own IP backbones) reach Tier 1 via "**Transit**" links and reach each other via "**Peering**" at **Internet Exchange Points (IXPs)**.
- **Tier 3** networks are **local ISPs**, either **multi-homed** (multiple Tier 2 uplinks for redundancy) or **single-homed**.
- At the edge are the **end users** (businesses, consumers).

ARIN is a nonprofit, member-based body administering IPs and **Autonomous System Numbers (ASNs)**; its site provides the central "**Site or Whois**" search for tracking network-block ownership.

Tools

Executed via **web WHOIS portals** or the CLI **whois** utility; accelerated with **allwhois** and **uwhois**, GUI tools **SuperScan** and **NetScan Tools Pro**, and web services like **CentralOps.net** (its **Domain Dossier** produces a full report). ARIN's portal handles North-American IP blocks.

Public-database (WHOIS) countermeasures

- Keep **administrative contacts** meticulously current, or better, **anonymise** them with the registrar's **privacy-protection** service.
- **Rigidly authenticate** any update to domain records to prevent **domain hijacking** — use **strong passwords or PGP**, and **never** rely merely on the

e-mail **"FROM"** field.

- *Classic failure:* in 1998 a hacker hijacked AOL's domain by spoofing an e-mail, redirecting massive traffic.

Limitation. Beyond IP spoofing, modern **GDPR/privacy regulation** now obscures much **WHOIS data by default**, reducing what an attacker can harvest.

1.7 Step 5 — DNS Interrogation

The attacker now extracts **internal-structure** information by deeply interrogating the target's DNS servers, aiming to **map domain names to IPs**.

Name resolution and the SOA record

Resolution relies on the **Start Of Authority (SOA)** record. In a typical scenario, the OS **stub resolver** queries a **Local DNS Server** ("Where is `www.ccsf.edu` ?"). Not knowing the answer, the local server asks the **root server** (`.`) for the SOA of `edu` , then the `.edu` server for the SOA of `ccsf.edu` , then finally the `ccsf` nameserver for the IP of `www.ccsf.edu` .

Zone transfers (the core abuse)

A **zone transfer** is a legitimate function that **synchronises DNS records from a primary to a secondary/backup server**. Due to **misconfiguration**, an untrusted user can sometimes request a **full zone transfer**, tricking the server into **dumping all its DNS records** — including **internal hostnames and IPs** — straight to the attacker.

Tools

Mapping scripts/tools: `dnsrecon` , `dnsenum` , `dnsmap` , `fierce` , `host` ; the classic `nslookup` extracts specific resource records.

DNS record types hunted during interrogation

- **DS (Type 43, RFC 4034)** — Delegation Signer; identifies the **DNSSEC signing key** of a delegated zone.
- **HINFO (Type 13, RFC 8482)** — Host Information. **Historically leaked exact OS and CPU type** (a major footprinting prize); RFC 8482 now

mandates minimal responses to "ANY" queries.

- **HIP (Type 55, RFC 8005)** — Host Identity Protocol; separates the **endpoint-identifier** and **locator** roles of IP addresses.
- **IPSECKEY (Type 45, RFC 4025)** — key record for IPsec.
- **KEY (Type 25, RFC 2535/2930)** — originally general-purpose; now limited to DNSSEC, and specifically only **SIG(0) (RFC 2931)** and **TKEY (RFC 2930)** (DNSKEY replaced it for DNSSEC, IPSECKEY for IPsec).
- **KX (Type 36, RFC 2230)** — Key Exchanger; identifies a key-management agent for some crypto systems (not DNSSEC); informational status, limited deployment.
- **LOC (Type 29, RFC 1876)** — Location; specifies a domain's **geographical location**.
- **MX (Type 15, RFC 1035/7505)** — Mail Exchange; maps a domain to its **mail servers (MTAs)**.

Executing the interrogation

Using `nslookup`, the attacker confirms the default server (e.g. `ns1.example.com` at `10.10.20.2`), queries `192.168.1.1` and finds its internal name `gate.example.com`, then attempts a **full zone transfer**:

```
> set type=any
> ls -d example.com. > /tmp/zone_out
```

Reading `zone_out` gives a massive advantage — it reveals **internal subnets and host configs**. For example, IP `192.168.230.3` mapped to A-record **acct18**, with an **HINFO** record exposing it as a **"Gateway2000" machine running "WinWKGRPS."** Even worse, **TXT** records can pinpoint physical locations (e.g. *"Location: Telephone Room," "Location: Library"*). To sift hundreds of records, attackers use **grep**, **sed**, **awk**, **perl** — e.g. `grep -i test /tmp/zone_out | wc -l` might reveal **96 lingering test servers**.

DNS countermeasures

- **Restrict zone transfers** so only **authorised secondary servers** can perform them (for **BIND**, configured in `named.conf`).
- At the perimeter, configure the firewall to **deny unauthorised inbound connections to TCP port 53** — standard queries use **UDP 53**, whereas a **full zone transfer requires TCP 53**.

- Ensure **external-facing DNS servers do not serve internal DNS info**, and **discourage verbose HINFO records**.
-

1.8 Step 6 — Network Reconnaissance

The final step builds an accurate **network-topology and access-path diagram**, identifying the exact route data takes from the internet, through intermediate nodes (routers, firewalls), to the target's internal servers.

Mechanism (TTL / ICMP)

The technique **abuses the IP header's TTL (Time-To-Live) field** (originally meant to prevent routing loops):

- The attacker sends a packet with **TTL = 1**. The first router decrements it to 0, **discards it**, and replies with an **ICMP Time Exceeded (Type 11)** error — **revealing its own IP**.
- The attacker repeats with **TTL = 2, 3, ...**, discovering each successive router hop by hop until the final target is reached.

Primary tools — note the important OS difference:

- **traceroute** (Linux/UNIX/Mac) — by default sends **UDP** packets to high ports (typically starting at **33434**).
- **tracert** (Windows) — by default sends **ICMP Echo Request** packets.
- **Advanced/graphical: VisualRoute**, McAfee's **NeoTrace**, Foundstone's **Trout**, **OWASP AMASS** — some combine tracing with **GeoIP** and **AS (ASN)** identification.

Reading the output

Example: `$ traceroute example.com` (resolves to `192.168.1.7`, max 30 hops, 38-byte packets). The output lists every hop with latency. The attacker identifies **border routers** and **infers firewall placement**: a sudden block of non-responses (shown as `* * *`) or an unexplained **latency spike** between two nodes (e.g. hop 7 jumping to **218 ms**) signals a **firewall / stateful-inspection device or packet filter**, before the trace finally reaches the destination (e.g. at hop 11).

Preconditions

Intermediate routers and the target firewall must **not** be purely stealth — they must be willing to **generate and route back ICMP error messages** to the attacker.

Countermeasures

- Deploy an **IDS (Snort, Bro)** to catch the **sequential TTL-decrementing probes** characteristic of traceroute sweeps.
- Configure **border/edge routers** with **ACLs** to **rate-limit or drop** inbound **ICMP/UDP** and outbound **ICMP Time Exceeded**, so external users cannot freely map the network.

Limitation. In modern enterprise networks traceroute is increasingly **ineffective**: admins disable ICMP responses ("**ICMP blackholing**") on intermediate routers to hide the DMZ's logical topology, leaving traceroute **blind from the outside**.

Phase Summary

Footprinting is tedious but essential work for both **attackers casing a target** and **defenders auditing their own perimeter**. Because of the sheer data volume, both sides **automate** with **shell, Python and Perl** scripts. For the defender, the overriding goals are **continuous vigilance, minimising information leakage at all times, and strict monitoring** to catch reconnaissance before it escalates into a full breach.

CHAPTER 2 — Scanning

Scanning is the phase that maps the target environment. It is built on **four objectives**, in order:

1. Determine **if a system is alive** (host discovery).
 2. Determine **which services are running / listening** (port scanning).
 3. **Detect the target's operating system**.
 4. Efficiently **process and store** the resulting scan data.
-

2.1 Determining if the System is Alive: Host Discovery

Host discovery (the **ping sweep**) exists only to answer *"is this host up?"* — it does **not** look at ports (that is the next phase). The method chosen depends entirely on whether the attacker is on the **same subnet** as the target or on a **remote network**.

Ping Sweep vs Port Scanning: the **ping sweep** maps *which machines are alive* across a network; **port scanning** then interrogates *one specific live host* to find *which ports/services* are open.

ARP host discovery (same subnet)

On the local subnet, **Address Resolution Protocol (ARP)** is the standard. The attacker broadcasts ARP requests and maps the network at the **data-link layer** from the replies.

- **Tool: `arp-scan`** — must run as **root** (`sudo`) to read MAC addresses; extremely fast. Example: `arp-scan --interface=wlan0 --localnet`. A documented scan of 256 hosts (`arp-scan 1.9`) received 29 replies in **~2.26 s** (**113 hosts/s**).
- **Information collected / impact:** an accurate IP map **plus the IP ↔ MAC pairing**, which reveals the **hardware vendor** — a crucial detail for choosing later exploits. Example mappings: `10.0.1.30` → **Dell**, `10.0.1.57` → **VMware**, `10.0.1.253` → **Cisco**.
- **Precondition:** the attacker must have logical access to the **same local segment (subnet)** as the target.

ICMP host discovery (remote hosts/routers)

For remote targets, the attacker uses **ICMP**, primarily the packet types **ECHO REQUEST**, **ECHO REPLY**, **TIMESTAMP**, and **ADDRESS MASK**.

- Standard `ping` uses **ECHO REQUEST/REPLY**. Example: `ping -c 2 192.168.1.1` sends two 64-byte requests; an active host replies (e.g. TTL 64, times 0.149 ms / 0.091 ms, 0% loss).
- **Superscan** sends many ICMP requests **in parallel**.
- **Hping3** / **nping** allow **any combination of flags on varying packet types**, including **spoofing MAC and IP addresses**.

Nmap host discovery

Nmap is the industry standard for both host and service discovery, with options for ICMP ping, address mask, timestamp, ARP ping, and TCP ping (often targeting web ports 80/443).

- **-sn** — skip port scanning (host discovery only); Nmap combines ARP ping and ICMP/TCP probes as needed.
- **-PR** — ARP ping.
- **-PE** — ICMP ECHO ping.
- **--send-ip** — with **-sn -PR --send-ip <range>** performs ARP ping plus some TCP/ICMP pinging.
- **sudo nmap -sn -PE --send-ip 192.168.1.1** skips **both** port scanning **and** ARP resolution — it returned the host up (0.060 s latency) **bypassing the ARP requirement even though the MAC stayed unknown.**
- **-Pn** — used to probe explicitly (e.g. only port 22).
- **Cain** (Windows-only) extends beyond standard host/service discovery.

TCP/UDP host discovery (when ICMP is blocked)

When ICMP is filtered, discovery falls back to **probing standard TCP/UDP service ports** (e.g. **port 80/HTTP**) to provoke a response. **Servers** are easy this way; **desktops are harder** — local firewalls frequently **ban inbound connections**, so a desktop may only be reachable via remote-desktop/file-sharing services, or if its local firewall is inadvertently disabled. Tools (Nmap, Superscan, Nping) can scan all ports or targeted ports, **but scanning all ports is slow and noisy.**

2.2 Ping Sweep Risks and Countermeasures

Generating extra discovery traffic can trip an **IDS alert**, so understanding the tools' behaviour matters.

Detection.

- Monitor with an **IDS (Snort)** or commercial network/desktop firewalls, which spot **repetitive ICMP/TCP/UDP patterns from a single source.**
- Host-based tools: **Scanlogd, courtney, ippl, protolog.**
- Manual log review ("eyeballs count") remains a vital layer.

Prevention.

- Use firewall **ACLs** to strictly limit inbound **ICMP**. Allow only necessary types — **ECHO_REPLY**, **HOST UNREACHABLE**, **TIME EXCEEDED** — into specific **DMZ** hosts, and ideally restrict even those to the **ISP's** own **IP** addresses.

Why ICMP is dangerous (the Loki case). ICMP's simplicity and general lack of security awareness made it a viable attack vector, per the SANS paper *"ICMP Attacks Illustrated."* **Phrack Magazine Issue #49 (1996)** documented **"Project Loki: ICMP Tunneling"** — the **Loki2** project showed how to backdoor an OS and covertly tunnel data inside **ICMP ECHO** packets. The countermeasure is **Pingd**, which moves **ICMP** processing out of the kernel and into user space.

2.3 TCP/IP Fundamentals (needed for port scanning)

The 3-way handshake

1. Both sides start **CLOSED**. The server does a **"Passive Open"** — creates a **Transmission Control Block (TCB)** and moves to **LISTEN**. The client does an **"Active Open"** (creates a TCB), sends **SYN**, and moves to **SYN-SENT**.
2. The server receives the **SYN**, replies **SYN+ACK**, and moves to **SYN-RECEIVED**.
3. The client receives **SYN+ACK**, sends the final **ACK**, and moves to **ESTABLISHED**; on receiving the **ACK** the server also moves to **ESTABLISHED**.

The TCP header

A 32-bit-wide structure: a 16-bit **Source port** + 16-bit **Destination port**; a 32-bit **Sequence number**; a 32-bit **Acknowledgment number**; then **Header length** (4 bits), a **Reserved** section, the **TCP flags** (**U**, **A**, **P**, **R**, **S**, **F**), and a 16-bit **Window size**; then a 16-bit **checksum** + 16-bit **Urgent pointer**; finally variable-length **Options**, **Padding**, and the **Data** payload.

The TCP flags

- **URG** — the Urgent-pointer field is significant.
 - **ACK** — the Acknowledgment field is significant; **every packet after the client's initial SYN must have this set.**
 - **PSH (Push)** — push buffered data immediately to the receiving application.
 - **RST** — reset the connection.
 - **SYN** — synchronise sequence numbers; **should only be set on the first packet from each end.** Its presence/absence alters the meaning of other flags/fields.
 - **FIN** — marks the last packet from the sender.
-

2.4 Determining Which Services Are Running: Port Scanning

Port scanning manipulates TCP flags (and UDP behaviour) to find which services (daemons) are listening. Each port resolves to one of three states: **Open, Closed, or Filtered.**

Critical privilege rule. Any scan that **forges packets via raw sockets** — **SYN, FIN, NULL, Xmas, ACK, Window** — requires **root/Administrator** on the *source* host. Only the **Connect scan** works as an unprivileged user (it uses the OS `connect()` syscall).

TCP Connect scan — **-sT**

Uses the OS's native `connect()` syscall to complete the **full 3-way handshake**, then closes politely. Works for any user, but is **noisy and easily logged** by the target application. (*Nmap default when not root.*)

TCP SYN / half-open scan — **-sS**

The stealthier default (when root). Forges packets with **raw sockets**: sends **SYN**, waits for **SYN/ACK** (= open) or **RST/ACK** (= closed), then **tears the connection down with an RST** instead of the final ACK — so the target application **never records a completed connection.** (*Nmap default when root.*)

- *Example:* `sudo nmap -sS 192.168.1.231` ran in 6.77 s, ignoring 994 closed ports, and found a **Fuji-Xerox** device (MAC `08:00:37:AD:D3:62`) with open

ports 80 (http), 139 (netbios-ssn), 445 (microsoft-ds), 515 (printer), 631 (ipp), 9100 (jetdirect).

The stealth scans: **FIN -sF** , **NULL -sN** , **Xmas -sX**

All three exploit a **loophole in RFC 793** governing how a host must react to *unsolicited* packets:

- **Closed port** → responds with **RST**.
- **Open (or filtered) port** → no response at all.

They differ only in flags: **FIN scan** sets only **FIN**; **NULL scan** sets no flags; **Xmas scan** sets **FIN + URG + PSH** (lit up "like a Christmas tree"). They establish **no handshake**, so they operate in stealth.

Biggest limitation (very important in enterprise): FIN, NULL and Xmas scans do NOT work against Windows. Microsoft's TCP/IP stack deliberately violates **RFC 793** and always replies with **RST** regardless of whether the port is open or closed — so every port falsely appears **closed**, making the scan useless.

ACK -sA and **Window -sW** scans

- **ACK scan** is used **not to find open ports but to map the firewall**: if the OS replies with **RST**, the packet passed unobstructed (port **unfiltered**); no reply means **filtered**.
- **Window scan** exploits **TCP Window-size anomalies** that some stacks (e.g. **AIX, FreeBSD**) show in their **RST** responses, letting the attacker distinguish **open vs closed** ports.

RPC scan and UDP scan

- **TCP RPC scans** map distinct architectures.
- **UDP scan (-sU)** sends empty UDP packets: a **closed** port replies with **ICMP Port Unreachable (Type 3, Code 3)**; an **open** port usually stays silent.
 - *Limitation*: notoriously **slow and unreliable** — firewalls that drop ICMP cause **false positives** (lost packets are mistaken for open ports).

Countermeasures. Deploy **next-gen firewalls with Stateful Packet Inspection** — a stateful firewall remembers established connections and **silently**

drops any **FIN/NULL/ACK** packet that doesn't belong to a completed **3-way handshake**. Add **IPS/Snort** rules that fire on TCP-flag anomalies (e.g. many consecutive SYNs with no ACK), and **close or filter every non-essential port**.

2.5 Detecting the Operating System

Knowing the exact OS (and Service Pack) is the **mandatory precondition** for a **successful exploit** — firing the wrong Metasploit payload can **BSOD** the victim and alert the Blue Team. Detection is **active** or **passive**.

Active OS detection

Banner grabbing — the simplest form. Services like **FTP**, **telnet**, **SMTP**, **HTTP**, **POP** often advertise their OS/version directly in the service banner.

Port-based guessing — some services are OS-specific:

- **Windows: 135** (Microsoft EPMAP / DCE RPC — remote management of DHCP/DNS/WINS), **139** (NetBIOS, sometimes restricted to Win 95/98), **445** (SMB, SAMBA-compatible), **3389** (RDP).
- **UNIX: TCP 22** (SSH), **TCP 111** (RPC portmapper — the UNIX equivalent of port 135), **TCP 512–514** (Berkeley "r" services like rlogin), **TCP 2049** (NFS), and high ports like **3277x** (RPC in Solaris).

Active stack fingerprinting — exploits the fact that **different vendors implement the TCP/IP RFCs slightly differently**. Nmap's **-O** compares responses against its signature file **nmap-os-fingerprints**.

- *Example:* `sudo nmap -O 192.168.1.17` (3.64 s) found ports **135** (msrpc), **139** (netbios-ssn), **445** (microsoft-ds), **3389** (ms-term-serv), **4445** (upnotifyp), **14000** (scotty-ft) and concluded **Microsoft Windows XP (SP2 or SP3)**, **1 hop away**. (Users can submit corrections to improve the database.)

Probe types used: the **FIN probe** (Windows 7/200x/Vista uniquely reply **FIN/ACK**); the **Bogus-flag probe** (undefined flags, which Linux answers); **Initial Sequence Number (ISN) sampling** for patterns; the **"Don't Fragment" (DF) bit**; the **TCP initial window size**; the **ACK value** (+0 or +1); **ICMP message quenching** (RFC 1812 error-rate limits), **quoting**, and

echoing integrity (whether IP headers were altered in ICMP errors); the **TOS** field (defaults to 0 but varies); **fragmentation handling**; and **TCP options**.

The full **67-bit signature** combines: initial packet size (16), Initial **TTL** (8), **Window size** (16), Max segment size (16), Window scaling (8), DF flag (1), sackOK flag (1), nop flag (1). **In practice, checking just the Initial TTL and Window size is often enough** to identify the system.

Passive OS detection

To stay **invisible to an IDS**, the attacker only **listens** — no packets injected. Passive fingerprinting runs from a central point or a **port-mirroring / SPAN** interface with packet-capture (PCAP).

- **Tools: Siphon** (passive port-mapping / OS-ID / topology tool, using signatures in **osprints.conf**) and **p0f** (the definitive CLI passive fingerprinter). It evaluates the same session attributes — **TTL, Window size, DF**.
- *Worked example:* an attacker telnets from **shadow** (192.168.1.10) to **quake** (192.168.1.11). A Snort capture on port 23 shows: **TTL 255, TOS 0x0, ID 58934, DF set, Window 0x2798, MSS 1460**. Searching the signature DB (`grep -i solaris osprints.conf`), whose rules are formatted **Window:TTL:DF:OS** (DF 1 = ON), matches **2798:255:1:Solaris 2.6-2.7** . Running Siphon (`siphon -v -i xl0 -o fingerprint.out`) confirms Host 192.168.1.11 on port 23 (TTL 255, DF ON) is **Solaris 2.6 or 2.7**.

Precondition for passive: strategic positioning — the attacker must be on the same LAN (e.g. via ARP spoofing) or have access to a **switch SPAN/mirror port** to see the traffic.

Limitations. Passive detection **fails** if applications build **custom packets**, if the attacker simply **cannot capture** traffic, or if a host **dynamically changes its connection attributes** (this last also defeats active detection). It also **fails through a NAT device** that rewrites IP headers (overwriting the original TTLs). Passive is **slow** and depends on the target actively "talking"; active is **fast but very noisy**, flooding security logs.

Countermeasures (both methods)

Deploy **Snort / Scanlogd** to detect anomalous scan options (irregular SYN packets, invalid flag combinations). Prevent by **filtering through secure**

application proxies or stateful firewalls that absorb and "normalize" the attacker's packets and reply on the server's behalf, hiding the real stack — or via **Active Defence**. Deliberately altering the OS stack to fool scanners is **not recommended** (it can break normal network communication).

2.6 Processing and Storing Scan Data

Efficiency matters because attackers must compromise many systems quickly. **Metasploit** provides the platform (tools, payloads, exploits) and handles data via a **PostgreSQL** database.

- **db_connect** sets the database interface: `msf > db_connect postgres:<password>@localhost:<port>/msf3`.
 - **db_nmap** (requires **root**) runs Nmap from inside the framework, dumping results straight into PostgreSQL. It is **preferred over Metasploit's native scan modules**, which are significantly slower.
 - *Example:* `msf > db_nmap 192.168.1.0/24` (Nmap 5.51SVN) mapped 256 IPs in 19 s, found **21 hosts up**, and stored e.g. host 192.168.1.12 (0.0028 s latency; 997 filtered ports; open **80/http**, **443/https**, **2869/icslap**).
 - **Scanning outside Metasploit** and importing: export XML with **-oX** (`sudo nmap -O 192.168.1.0/24 -oX subnet_192.168.1.0-OS` — e.g. host 192.168.1.12 running **Linux 2.6.19/2.6.36, 0 hops**), then load with **db_import** (parsed via **Nokogiri**).
 - Once populated, **hosts** lists all targets with identified OS, and **services** lists all ports/services network-wide. The **-s** filter parses the environment fast — e.g. list all hosts with **SSH** active, or find all **Windows 2008** machines for a targeted attack.
-

Phase Summary

Scanning turns the passive footprint into an **active map**: which hosts are alive (host discovery), which services they expose (port scanning), and what OS underlies them (fingerprinting) — all stored efficiently for the exploitation phase. The recurring defensive themes are **stateful filtering**, **ICMP/port ACLs**, **IDS/IPS signature detection**, and **closing unnecessary services**; the

recurring attacker themes are the **trade-off between stealth and speed**, and the constant caveat that **evasive stealth scans break against Windows' non-RFC-compliant stack**.

CHAPTER 3 — Enumeration

3.1 Scanning vs Enumeration (and where Service Fingerprinting fits)

Scanning and enumeration are related but differ sharply in intrusiveness:

- **Scanning** is generic, operating at the **transport layer (Layer 4)** — it sends basic TCP/UDP probes (e.g. a SYN; a SYN/ACK back = open port) and does light banner grabbing.
- **Enumeration** is an **active, intrusive application-layer (Layer 7)** phase: it **establishes real connections** to discovered services and issues **directed, structured queries** — "give me the user list," "show me the shared folders." It is **platform-specific**, depending heavily on the prior port-scan and OS-detection results to choose the attack path.
- **Service Fingerprinting** sits between them: determining the **exact application and version** listening on a port.

Goal of enumeration: harvest weaponisable data — **valid user account names** (for password-guessing/brute-force), **misconfigured file shares**, and **old software versions** with known vulnerabilities (e.g. a web server unpatched against a remote buffer overflow). The attacker maps **port → protocol → service → software**, commonly targeting **FTP (TCP 21)**, **Telnet (TCP 23)**, **SMTP (TCP 25)**.

Key limitation: unlike scanning — which can be made partly stealthy (SYN stealth scan, fragmentation) — **enumeration is extremely noisy**. It generates obvious application logs on the target (e.g. *"anonymous connection attempt to the file server"*) and is easily caught by a competent Blue Team or IDS.

3.2 Service Fingerprinting and Vulnerability Scanners

Service fingerprinting

The attacker balances **stealth (manual interaction)** against **efficiency (automated tools)**. Nmap **-sV** is the standard. A plain scan uses **nmap-services** to map ports to their *default* assigned service; a version scan uses the **nmap-service-probes** database, actively sending queries and analysing responses to find the **true protocol and version**.

- *Example (hidden service)*: an admin hides OpenSSH on **TCP 1417**. A basic scan `nmap -sS target.com -p 1417` misreports it as `timbuktu-srv1` (purely by port number). A version scan `nmap -sV target.com -p 1417` interacts with the port and correctly exposes **ssh running OpenSSH 3.7**.
- **Amap** provides a second opinion using a distinct pattern-matching technique.

(Because every daemon reacts slightly differently to malformed/probe packets — a consequence of how its source was written — analysing the response reveals the exact identity/version even when the banner is hidden.)

Vulnerability scanners

Once versions are fingerprinted, scanners cross-reference **massive databases of known vulnerability signatures (CVEs)** and **actively inject non-destructive test payloads** to *confirm* whether a logic flaw, default password, or buffer overflow is physically present (rather than merely guessing from the banner).

- **Landscape**: Nessus (free/open-source **until version 3**, then proprietary closed-source; maintained by **Tenable**, supports custom plug-ins in **NASL**), **OpenVAS** (open-source), plus commercial tools from **McAfee**, **Qualys**, **Rapid7**, **nCircle**, **Tenable**.
- **Output/impact**: a surgical report classified by **severity (Critical/High/Medium/Low)**, pointing to the **exact exploit module (e.g. in Metasploit)** to launch for a root shell.
- **Nmap vs Nessus**: Nessus is focused and powerful for vuln assessment; **Nmap** has **wider scope** and bridges the gap with the **Nmap Scripting Engine (NSE)** — using the **Lua** language and a large library of scripts for discovery, version detection, backdoor detection, and even **direct exploitation**.
- **Countermeasures**: constant patch/configuration management; audit yourself regularly; configure **IDS/IPS** to recognise scanner signatures (e.g. Nessus) and **block or intentionally slow** them, redirecting attackers to softer targets.
- **Limitations**: vulnerability scanning is **monstrously noisy** (alerts in any competent SOC), and aggressive malformed probes can cause **accidental**

DoS/crashes on fragile legacy systems.

3.3 Basic Banner Grabbing

Banner grabbing captures the **initial text a server returns on connection**, frequently leaking **vendor, version, and hints of known vulnerabilities**.

- **Manual:** `telnet www.example.com 80` may return an **HTTP 400 Bad Request** — yet the header still advertises **Server: Microsoft-IIS/5.0** with `content-type/date`.
- **Automated (netcat):** fed a file of directed requests to elicit richer responses — e.g. `nc -nv -o banners.txt 10.219.100.1 80 < nudge.txt` pulls a full **HTTP 200 OK**, confirming **Microsoft-IIS/5.0**, revealing it runs **ASP.NET**, and exposing active session cookies (**ASPSESSIONID...**).
- **Countermeasures:** shut down unnecessary services entirely, apply **ACLs**, audit regularly, and **disable/obfuscate vendor & version data** in banners.

3.4 Enumerating Common Network Services

FTP Enumeration (TCP 21)

FTP is popular for content upload but has a fatal flaw: **passwords travel in cleartext**. Attackers target **anonymous FTP** (login `anonymous` , e-mail as the fake password), searching indices like `ftp-sites.org` or the **Google dork** **`intitle:"Index of ftp://"`** . Once in, standard commands (`dir` , `ls` , `cd` , `get`) navigate and download.

- *Example:* an anonymous server's web index (`http://www.tmk.com/ftp/`) exposes folders (humor, tmk-software, vms-freeware) and an **overly informative HTTP banner:** Apache/2.0.63 (FreeBSD), PHP/5.2.5-Suhosin, mod_ssl/2.0.63, OpenSSL/0.9.8g, mod_perl/2.0.3, Perl 5.8.8. Direct FTP (`ftp ftp.tmk.com`) confirms **MultiNet FTP Server Process U5.2(16)** and permits anonymous login.
- **Information/impact:** system backups, website source, unprotected databases, and cleartext-password sniffing when other users log in legitimately on the same network.
- **Countermeasures:** abandon FTP for **SFTP (over SSH)** or **FTPS (over SSL)**; serve public content over HTTP; watch aggressively for anonymous FTP and **disallow unrestricted uploads**.

Telnet Enumeration (TCP 23)

Telnet broadcasts system banners before login, revealing OS/version/vendor. It also enables account enumeration by brute force: the attacker tries usernames and reads the error messages — distinguishing "invalid username" vs "invalid password" compiles a definitive list of valid accounts. Like FTP, all passwords/data are cleartext.

- **Countermeasures:** replace with **SSH**; if legacy Telnet is mandatory, restrict to authorised source IPs or a VPN tunnel; modify banners and enforce reconnection delays between failed logins.

SMTP Enumeration (TCP 25)

SMTP can be enumerated manually via Telnet to validate accounts and mail structure, using two commands:

- **VRFY** — confirms whether a specific username is valid.
- **EXPN** — expands aliases/ mailing lists to reveal the real delivery addresses behind them.

Example: connecting to mail.bigcorp.com:25 yields the **ESMTP Sendmail 8.8.7** banner; **vrfy root** returns **250 root <root@bigcorp.com>**; **expn adm** validates the admin alias. Scaled with **vrfy.pl** (target server + username wordlist).

- **Impact:** a validated list of real users/admin aliases — the fuel for brute-force and spear-phishing (often APT step #1).
- **Countermeasures:** disable **EXPN** / **VRFY** or restrict them to authenticated users (natively configurable in modern Sendmail and Exchange).

DNS Enumeration (TCP/UDP 53)

DNS normally runs on **UDP 53**; **TCP 53** handles zone transfers. A misconfigured server allows a **zone transfer** dumping the domain's entire zone (critical **A** and **HINFO** records). CLI: **nslookup (ls -d <domain>)** and **dig** — e.g. **dig @10.219.100.1 version.bind txt chaos** enumerates the exact **BIND** version.

DNS Cache Snooping examines only the resolver's **local cache** to learn what sites the org visits, using the **+norecurse** flag (ask if a record is already cached without a fresh lookup):

- Not visited → status **REFUSED** , **ANSWER 0**.
- After a standard recursive query the server fetches and caches the IP (**NOERROR**); subsequent **+norecurse** queries return **NOERROR** + the **cached IP**, confirming presence.

Automation: **dnsenum** (scrape Google for subdomains, brute-force subdomains, list network ranges, WHOIS them); **CentralOps.net** (Domain Dossier — domains/IPs/registrant/DNS in one report, plus Email Dossier, Ping, Traceroute, NsLookup, AutoWhois); **Fierce.pl** (attempts a zone transfer first, then seamlessly pivots to brute-forcing hostnames across subnets when transfers are disallowed).

- **Countermeasures:** separate internal and external DNS servers so internal targets are never public; **restrict zone transfers** to authorised machines at the border; **restrict queries** to prevent cache snooping.

TFTP Enumeration (TCP/UDP 69)

TFTP is inherently insecure: **cleartext and no authentication whatsoever**. The attacker just sends a **Read Request (RRQ)** to port 69; if the file exists and is accessible, the server sends it. Example: `connect 192.168.202.34 then get /etc/passwd /tmp/passwd.cracklater`. Because TFTP is heavily used by routers and VoIP phones to fetch firmware/config, **configuration files are the prime targets** (they often contain admin passwords, cleartext or crackable).

The defining limitation (FTP vs TFTP): TFTP has no **DIR / LS command** — the attacker works "blind" and must **know or guess the exact filename and path** (e.g. `router-config`) to retrieve it, making it less trivial than FTP.

- **Countermeasures:** wrap the service with **TCP Wrappers** (software firewall — only approved clients); restrict access internally to the **/tftpbboot** directory (chroot); **block TFTP at the border firewall** (never exposed to the Internet).

Finger Enumeration (TCP/UDP 79)

If enabled, **Finger** freely displays currently logged-in users with **no authentication**. `$ finger` (or `finger user@host`) lists **login name**, **real**

name, TTY, idle time, and exact login timestamps — valuable for targeted social engineering and building brute-force wordlists.

- **Countermeasure: completely block/disable remote Finger** — a "leaky" service to switch off on principle (like `rpcbind`). (*Practically absent on modern systems; effective only against legacy UNIX.*)

HTTP Enumeration (TCP 80)

Starts with banner grabbing via `telnet` / `netcat` (for SSL, redirect through an `sslproxy` or use `openssl`), then relies on **aggressive crawling/spidering**:

- **Sam Spade** (Windows GUI) mirrors an entire site locally, with options to fetch specific file types (`.html/.asp/.txt`), include headers/inline images, and scan source for **e-mail addresses, hidden form values, and custom regexes** (e.g. "password", "admin").
- **Grendel-Scan** slowly crawls and reports vulnerabilities, parsing for **developer comments, robots.txt, and hidden directories**, testing logical 404 responses, running **Nikto** CGI searches, and requesting many extensions (`.cgi, .aspx, .jsp, .pl, .dll, ...`).
- **What it seeks/impact: robots.txt** (a map of the dirs the admin *doesn't* want indexed — i.e. sensitive targets); exact **software versions** (→ CVE lookup); **session cookies** identifying the tech stack — **ASPSESSIONID** (classic ASP), **JSESSIONID** (Java/J2EE) — to plan **session hijacking**; and **backup/test files** (`.bak, .old, .inc`) leaking source and cleartext DB passwords.
- **Countermeasures**: change/fake default banners (to fool automated malware); because **IIS** has many ready exploits, deploy **MS URLScan (IIS v4+)** for filters/custom banners; remove comments/test files/sample dirs/backups from production; deploy a **WAF** with rate-limiting.

MSRPC Enumeration (TCP 135)

The Microsoft RPC **endpoint mapper (portmapper)** on TCP 135 acts as a "switchboard": querying it lists which **applications/services** are bound to which dynamic ports, identified by **UUID**. Tools: `epdump` (Windows Resource Kit) — e.g. `epdump 192.168.189.1` reveals a binding

`ncadg_ip_udp:192.168.0.1[1028]` annotated 'Messenger Service';

Linux/Backtrack uses `rpcdump.py` (Impacket) with near-identical annotated UUID results.

- **Countermeasures:** block port **135** at the firewall if possible; modern networks avoid exposure via **VPN**, **Outlook Web Access (OWA)** over **HTTPS**, and **RPC** over **HTTP** (Exchange 2003+). (*Limitation: raw UUID output must be manually mapped to known services.*)

NetBIOS Name Service (UDP 137) and NetBIOS Session (TCP 139)

NetBIOS Name Resolution

NBNS on **UDP 137** is Microsoft's legacy, DNS-like naming service, **IPv4-only**. When a command references a computer by name (e.g. `PING SAMP4`), Windows must resolve the name to an IP. Though Windows prefers DNS, it still uses NBNS across all versions. The resolution order checks: **local host name** → **local Hosts file** (loads into resolver cache on XP/2003) → **DNS server** → **DNS resolver cache** → **RAM NetBIOS name cache** → **WINS** (NetBIOS name server) → up to three broadcast Name Query Requests on the local subnet → local `Lmhosts` file.

Attackers use **NET VIEW** to list domains/computers. NBNS is normally local-only, but can be **routed over TCP/IP for remote enumeration**. Tools:

NLTEST / **NETDOM** (locate domain controllers), **NETVIEWX** (locate services), **NBTSTAT** (single host, e.g. `nbtstat -A <IP>`), and **NBTSCAN** / **NMBscan** (scan whole ranges, dumping the full NetBIOS name table — revealing **UNIQUE Workstation Services**, **File Server Services**, and **MAC addresses**, which also expose the machine's role: Workstation / File Server / Domain Controller).

- **Countermeasures:** block **UDP 137** (or restrict to authorised hosts); disable **Alerter** and **Messenger** services so user data doesn't appear in dumps. (*Caveat: blocking UDP 137 disables NBNS name authentication and can break dependent legacy apps.*)

NetBIOS Null Sessions (TCP 139 / 445)

NetBIOS Sessions on **TCP 139** enable the notorious "**Null Sessions**" (Anonymous Logon): **SMB hands out information freely**. Null Sessions are **off by default on XP** and later, **unavailable on Win 95/98/Me**, but **wide open by default on Windows 2000 and NT**.

- A plain `net view \\192.168.11.29` may return "**System error 5: Access is denied.**"

- Establishing a null session with **empty credentials** — `net use \\192.168.11.29\IPC$ /user:""` — connects to the hidden **IPC\$** (Inter-Process Communication) share and **bypasses the restriction**. Subsequent `net view` then enumerates **shares, user accounts, and password policies**. Under the hood the attacker invokes native APIs like **NetUserGetInfo** and **NetShareEnum**.
- **GUI: Somarsoft DumpSec** (formerly DumpAcl) visually enumerates permissions/services and exposes **PswdRequired**, **PswdExpires**, **LastLogonTime**, and driver/service run status.
- **Registry enumeration** via null session is **not** possible — remote `reg /DumpSec` registry access requires **Administrator** by default. (*Gary McKinnon infamously leveraged remote registry access to breach the Pentagon.*)
- **SID/RID trick**: a **SID** uniquely identifies a security principal; its final segment is the **RID**. Default RIDs are static: **Administrator = 500**, **Guest = 501**, first user accounts = 1000, 1001, ... Tools `user2sid` / `sid2user` extract names/SIDs remotely — dangerous because they **bypass RestrictAnonymous = 1** and can **locate the Administrator account even if renamed**. (*These work against the NT family but fail against Windows XP SP2.*)
- **All-in-one: Wininfo, Winfingerprint** (queries via Active Directory & WMI), **NBTEnum** — automate the null session and dump primary domains, password policy (max age, min length), lockout duration, and logged-in users.
- **Countermeasures**: block **TCP 139 and 445** at the perimeter; via `regedt32` set **RestrictAnonymous** to 1 (or 2 on Win 2000+) at **HKLM\SYSTEM\CurrentControlSet\Control\LSA**. (*But NBTEnum / Userinfo can still bypass this via the NetUserGetInfo API at Level 3; and since Anonymous-Access settings don't cover remote registry, the registry must be independently locked down. Audit yourself with DumpSec.*)

3.5 Advanced Infrastructure Enumeration

SNMP Enumeration (UDP 161)

SNMP ("Security Not My Problem") is meant for network management, so it inherently exposes **granular internal information** on devices, software, and systems. Its minimal security uses **Community Strings** — effectively **plaintext passwords** — in three types: **Read-Only**, **Read-Write**, and **Trap** (Trap rarely used). Because they're frequently left at factory defaults ("**public**" = read,

"private" = read-write), attackers guess them or simply sniff them with Wireshark.

Data lives in the hierarchical MIB (Management Information Base) tree (like the Windows Registry), each leaf an **OID**: root branches **iso(1) → org(3) → dod(6) → internet(1)**, then **mgmt(2)** (housing **ip(4)**, **tcp(6)**) and **private(4) → enterprises(1)** where vendors append hardware data (Microsoft stores Windows account names here); later the **SNMPv2** branch added **security(5)**, **snmpv2(6)**.

- **Tools:** **snmputil** (Windows NT Resource Kit), **snmpget** / **snmpwalk** (Linux net-snmp — **snmpwalk** walks the whole tree), and **SolarWinds IP Network Browser** (renders numeric OIDs as readable tables). Querying OIDs (e.g. **.server.svUserTable.svUserEntry.svUserName**) reliably dumps usernames, running services, share names/paths/comments, and domain names. The attacker sends **GET/GETNEXT** to read; a guessed Read-Write string allows **SET** to reconfigure devices or cause DoS.
- *Example:* SolarWinds scans **192.168.202.0/24**, queries NT host **192.168.202.33** with default "**public**", and lists local accounts (Administrator, Guest, lucca, mike) plus a share **C:\Binaries** commented "*Free goodies!*".
- **Escalation:** with the Read-Write string ("**private**"), on **Cisco** an attacker uses **copy-router-config.pl** to copy the full device config to an external **TFTP** server, then decodes passwords and launches **DoS**.
- **Countermeasures:** remove/disable unneeded agents; **change default strings**; block **TCP/UDP 161** at the perimeter; restrict agents to **authorised management console IPs** (e.g. Nagios/Zabbix); and, crucially, **migrate to SNMPv3** (strong authentication hashing + traffic encryption, absent in v1/v2).
- **Limitations:** **SNMPv1/v2** are **UDP-based** — packet loss on congested networks causes **false negatives**; a correctly configured **SNMPv3** target closes this vector entirely.

BGP (TCP 179)

Border Gateway Protocol on **TCP 179** is the de facto routing protocol between **Autonomous Systems**. Multi-uplink organisations use a unique **ASN** to route packets. Attackers query BGP (e.g. **via Telnet** to BGP-speaking routers, or a

looking glass) to **enumerate all networks announced under the target's AS-Number**, yielding a **wider pool of target IPs**.

- **Countermeasure:** essentially **none** — BGP is critical to core Internet routing and **cannot be blocked**. (*Limitation: only reveals publicly announced blocks, and requires the target to run its own AS — small orgs fall under the provider's ASN.*)

Active Directory LDAP (TCP/UDP 389 / 3268)

Windows AD uses **LDAP on TCP/UDP 389** (and the **Global Catalog on 3268**) to house **all user accounts, groups, and structural info** on domain controllers. If the domain runs in **legacy-compatible mode** (to support old systems like **Windows NT4 Server**), any **domain member** — or an **anonymous bind** — can use **ldp.exe** or the **Active Directory Administration Tool** to freely enumerate the entire directory.

- **Mechanism:** connect to LDAP (389) or Global Catalog (3268); via **anonymous bind** (in **ldp.exe** : Connection → Bind with empty credentials → View → Tree) or a low-priv account, send filters (e.g. **objectClass=user**) to **dump the whole directory tree**. Export en masse with **csvde / ldifde** to CSV/LDIF.
- **Information/impact:** the **lateral-movement treasure map** — exact group hierarchy (who's in **Domain Admins**), service accounts, topology, and critically the **Description** fields, where lazy admins often leave **temporary/cleartext passwords**.
- **Countermeasures:** filter **389/3268** at the perimeter; use **Native Mode** (e.g. Native Win 2000+), which **disables anonymous LDAP binds** by refusing NT4 backward compatibility; segregate DCs into isolated VLANs; enforce signed/encrypted **LDAPS (port 636)**. (*Modern Windows Server 2012+ disables unauthenticated LDAP by default — the attack then requires a compromised low-priv account first.*)

3.6 Summary and Conclusion

Enumeration exploits OS and application architectures to siphon data. Beyond the primary protocols, attackers also target **UNIX RPC (TCP/UDP 111 / 32771)** with **rpcinfo**, **rwho**, **rusers**; **SQL resolution (UDP 1434)** with **SQLPing**; **Oracle TNS (TCP 1521/2483)**; **NFS (TCP/UDP 2049)**; and **IPsec/IKE (UDP 500)**.

The defensive goal is to **"seal the lips"** of your software: aggressively disable/restrict notoriously leaky services (**finger** , **rpcbind**), scrub defaults (especially SNMP **"public"**), and apply a **layered defence** — patch holes, screen traffic tightly with firewalls, and, above all, **routinely audit your own perimeter with Nmap and Nessus**.

CHAPTER 4 — Hacking Windows

4.1 Why Windows Is Vulnerable

Windows' security problems stem from structural sources: **backward compatibility** (critical for business, enabled by default, but retains older, less secure code), and the constant **proliferation of new features** that expands the attack surface. Historically Microsoft issued **~70 security bulletins per year**, forcing admins to track resources like the **MSRC Security Update Guide** to prioritise patches. Risk is tied to **popularity and complexity** — the code base grew **tenfold from Windows NT 3.51 to Windows 7**. Notorious outbreaks include **Code Red, Nimda, Slammer (Sapphire), Blaster, Netsky, Conficker, EternalBlue, NotPetya** (Slammer infected **~74,855 hosts globally in 31 minutes**).

Newer versions added defences: **fewer default network services, a host firewall on by default, and User Account Control (UAC)**. The financial stakes are high — per **Zerodium**, a zero-click Windows RCE pays up to **\$1,000,000** (IIS RCE \$500k, Outlook/Exchange RCE \$250k, Windows LPE/Sandbox Escape \$80k).

4.2 Two Attack Categories

1. **Unauthenticated attacks** — remote network exploits with **no initial credentials**.
2. **Authenticated attacks** — an insider or already-breached attacker focused on **privilege escalation** (to Administrator/SYSTEM) and **password extraction**.

Unauthenticated attackers use **four vectors**: **authentication spoofing** (guessing/stealing passwords), **attacking network services**, **exploiting client software**, and **leveraging device-driver flaws**.

4.3 Authentication Spoofing: Remote Password Guessing

The prime target is **SMB (TCP 445/139)**, connecting to enumerated admin shares **IPC\$** or **C\$** (disabled by default on modern consumer Windows, but often exposed on **domain controllers**). Other targets: **MSRPC (TCP 135)**, **Terminal Services/RDP (TCP 3389)**, **SQL (TCP 1433 / UDP 1434)**, **Web/SharePoint (HTTP 80 / HTTPS 443)**.

Mechanism: build a **credentials.txt** of username/password pairs (often from default-password databases) and feed it into a **FOR loop + net use** against **IPC\$**. CLI tools: **enum**, **Brutus**, **THC Hydra**, **Medusa**, **Venom**; for RDP, **TSGrinder** or a brute-force-patched **Rdesktop**. A key hurdle is **account lockout** ("Logon failure") after too many attempts.

Countermeasures (defense-in-depth): firewall SMB on **139/445**; host **IPSec** filters + **Windows Firewall**; disable SMB if unused; enforce **strong/long passwords** via **domain policy**; set an **account-lockout threshold** (**SECPOL.MSC**) that also covers the **built-in Administrator**; enable **auditing of logon failures** and review Event Logs (parse with **Dumpel**, Win2000 Resource Kit); deploy **IDS/IPS**.

4.4 Eavesdropping on Network Password Exchange

Windows uses three authentication protocols: **LM** (transmits a hash), **NTLM** (encryption), and **Kerberos** (private/optional public-key encryption). On switched networks the attacker must first perform **ARP spoofing/poisoning** to redirect traffic, then sniff/crack with **Cain & Abel**, **LCP**, **L0phtcrack**, **KerbSniff**. Cain is the most widely used tool for sniffing **LM challenge-response** hashes.

Kerberos sniffing (the pre-auth flaw). The exchange runs Client→**AS** (returns **TGT** + session key encrypted with a key derived from the user's password, **K_C**)→**TGS** (returns client-to-server key)→**Service Server**. The critical vulnerability is in step 1: **Kerberos 5** sends a **pre-authentication packet** containing a **timestamp** encrypted with a key derived directly from the user's password. Attackers capture it with Cain's **MSKerb5-PreAuth** sniffer and **brute-force/dictionary-crack** it offline — if the password is weak,

it's revealed. Because it's offline, the only defence is **long, complex passwords** (and no account lockout is triggered).

Countermeasures: disable LM (NTLM is far harder to crack than LM); enforce complexity/no dictionary words; **encrypt traffic with IPsec** to render sniffed packets unreadable.

4.5 Man-in-the-Middle (MITM) and SMB Relaying

MITM actively **intercepts and relays** legitimate authentication to gain server access, using **ARP spoofing and DNS redirection** to force victims onto malicious SMB servers.

- **SMBRelay / SMBProxy** harvest username/hash from incoming SMB and **pass the hash onward** to authenticate to the target **masquerading as the victim** — no offline cracking needed.
- **SMB Credential Reflection & Forwarding** redirect tokens back at the victim or to third-party services.
- Tools: **Cain, Squirtle, SMBRelay3**. Cain **downgrades** clients to weaker dialects to record easily broken data (historically even breaking legacy XP/2003 RDP encryption).

Countermeasures: hard if the attacker is already on the LAN — **disable NetBIOS Name Services** (force DNS); enforce authenticated/encrypted protocols via **Group Policy**; the definitive architectural fix is **SMB Signing** (cryptographically binds auth to a session, making relay impossible); verify server identity via trusted certificates. (*Classic SMB relay fails entirely where SMB Signing is enabled.*)

4.6 Pass-the-Hash and Pass-the-Ticket

Pass-the-Hash (PtH) bypasses cracking entirely: (1) compromise one machine, (2) **dump the hashes from RAM**, (3) **inject the LM/NTLM hash** into the protocol to authenticate to other services. Because **NTLM natively accepts the hash as proof of identity**, one compromised machine can lead to full domain compromise — and crucially, **if a Domain Admin logged in before the attack, their privileged hash is still in memory** and can be stolen and replayed against the DC.

In-memory encryption. Windows protects credentials with `LsaEncryptMemory()` in `Lsasrv.dll` (encrypting the `NTLM_CREDS_BLOCK`): XP/2003 use **desX-CBC** or **RC4** (desX if `mod(size/8)==0`, else RC4); Vista/7/2008 use **3DES-CBC** or **AES-128-CFB**. The fatal flaw: the **decryption keys also live in RAM**, so a SYSTEM-level tool simply extracts the keys, decrypts, and steals the hashes.

Tools: **WCE** (`wce.exe`) dumps in-memory credentials (username, domain, LM & NT hashes) for current/future/terminated sessions (XP–2008); the modern standard is **Mimikatz** — `sekurlsa::logonpasswords` to dump, `sekurlsa::pth` to launch a process (e.g. `cmd.exe`) in a stolen user's context.

Pass-the-Ticket (PtT). Instead of a hash, the attacker steals a **Kerberos ticket (TGT/TGS)** from LSASS memory (a `.kirbi` file) and injects it to impersonate a user. **Mimikatz** `sekurlsa::tickets /export` then `kerberos::ptt <ticket.kirbi>`; the modern tool is **Rubeus**.

Key difference: an NTLM hash is valid forever (until the password changes), but a Kerberos ticket has a **TTL — 10 hours by default** — so a stolen ticket expires.

Countermeasures: NTLM is vulnerable by design (no patch) — so prevent the initial intrusion, enforce **2FA** (renders a lone hash/ticket useless), apply the **Tiering model** (forbid Domain Admins from logging into untrusted Tier-2 workstations so their creds never land in RAM), and deploy **Windows Defender Credential Guard** (hardware-virtualisation isolation of LSASS, inaccessible even to SYSTEM). PtH also fails in a strict **Kerberos-only** environment.

4.7 Remote Unauthenticated Exploits

These target flaws/buffer overflows/misconfigurations in Windows itself, needing **no credentials**. Targets: TCP/UDP network services, the device-driver interface, and user-mode apps (Office, IE, Adobe Reader).

Attackers use frameworks: **Metasploit** (open-source, huge module archive — usually a couple of months behind Microsoft alerts but effective on unpatched systems), or commercial **CORE IMPACT** / **Canvas** (pricier, sometimes zero-days). Workflow: **locate exploit module** → **customise parameters** → **select payload** (e.g. reverse shell) → **configure options** (target IP, IDS evasion) → **launch**. *Example:* `ms04_011_lsass` (LSASS overflow) spawns a reverse shell "Session 1" on the target. Metasploit's **Print Spooler** exploit was

famously used by **Stuxnet**, a cyber-weapon that used unauthenticated zero-days to destroy Iranian nuclear centrifuges.

Countermeasures: patch quickly (most critical); implement workarounds/disable affected services for zero-days; audit/log/monitor for exploit signatures; establish a **CSIRT** including infosec, IT, legal, **HR**, and **PR**.

4.8 End-User Application Exploits

When the perimeter is secure, **end-users are the weakest link**. Attackers trick them into opening malicious files or sites. Historic worst offenders: **Adobe Flash Player and Adobe PDF Reader**. Frameworks like Metasploit search for app vulns (`search/w adobe flash`) and embed payloads in benign-looking files.

Example: **CVE-2018-4878**, a Flash **use-after-free** (dangling pointer in the Primetime SDK), **CVSS v3 9.8 Critical**, exploited in the wild in early 2018 for arbitrary code execution.

Countermeasures: firewall to limit **outbound** connections; aggressive app patching; antivirus (emphasis on **e-mail attachments**); **Least Privilege** (never browse/read e-mail as Administrator); read e-mail in **plaintext** (block malicious HTML) and set Office **macro security** to highest.

4.9 Device Driver Exploits

Because Windows supports vast **plug-and-play** third-party drivers that run in **kernel mode (Ring 0)**, a single weak driver means **total system compromise**. A prime example is **wireless-driver buffer overflows**: broadcasting **oversized beacon frames** (Metasploit WiFi modules) from a rogue AP triggers code execution on every vulnerable machine in range — **no connection and no user interaction required**, merely parsing the beacon frame in kernel memory. An exploit failure causes a **BSOD**; success gives invisible **Ring-0 (SYSTEM)** control.

Countermeasures: apply vendor patches; disable wireless in high-risk areas; enforce **Driver Signing** (though Microsoft testing isn't foolproof); use the **User-Mode Driver Framework (UMDF)** to move drivers into user space (**Ring 3**), so bad drivers crash silently rather than compromising the kernel.

Limitation: radio/Wi-Fi driver attacks require **physical proximity** (radio range), unsuitable for remote Internet operations.

4.10 Authenticated Attacks & Privilege Escalation

Having a foothold as a Limited User/Guest, the attacker's goal shifts to **Privilege Escalation to Administrator or SYSTEM** (SYSTEM is more powerful than Administrator). Historic tools: **getadmin.exe** (DLL injection), buffer overruns like **MS03-013**. Notably, a standard Administrator can **schedule a task to run as SYSTEM** — still viable via **schtasks.exe** or the legacy **at** command.

Countermeasures: rigorous patch management; restrict interactive logon to trusted accounts via **secpol.msc** → **Local Policies** → **User Rights Assignment** → **Deny log on locally** (explicitly deny Guest).

4.11 Extracting and Cracking Passwords (Post-Exploitation)

With Admin/SYSTEM on one machine, the attacker grabs more hashes to move deeper (first **disabling the Windows firewall** for lateral movement).

Where hashes live (exact paths): local hashes are in the **SAM** at **%systemroot%\system32\config\SAM** and in the registry at **HKLM\SAM** (the **SYSTEM** file is also needed for the **Syskey** decryption key); in Win2000+ domains, hashes are in the AD database **%windir%\WindowsDS\ntds.dit** (whose compromise on a DC means the whole domain falls).

Extraction methods: (1) **boot an alternate OS** (Live CD/USB) to bypass file locks and copy SAM; (2) grab the **backup SAM** from the Repair Disk (often **SYSKEY**-encrypted); (3) **memory injection** — inject a DLL into **LSASS** on a running system with **pwdump2/3/4**, **Cain**, **Ophcrack**, or (the modern dominators) **Mimikatz** (**sekurlsa::logonpasswords**) and **gsecdump**; **Volume Shadow Copy (VSS)** can also bypass the live file lock.

Cracking / the salting flaw. Hashes are one-way. **NTLM is robust; legacy LM is weak** (LM off by default from Vista/7). The fatal flaw is **Windows does not salt NTLM hashes** — two accounts with the same password produce the **identical hash** (e.g. password → 8846F7EAAE8FB117AD06BDD830B7586C). Windows uses fast **MD4**; Linux/macOS salt (different **/etc/shadow** strings for the same password) and hash **slowly (thousands of rounds)**. The lack of salt makes **Rainbow Tables** (precomputed hashes, trading memory for time — LM tables once sold for **\$120**) lethal, alongside **John the Ripper** (CLI) and GPU tools like **Elcomsoft Distributed Password Recovery** (100×+ faster).

Countermeasures: enforce **long/complex passwords (>15 chars** defeats rainbow tables); add non-printable chars (**ALT+255**); use password managers;

disable LM via Group Policy; deploy **Credential Guard** (isolates LSASS) and **LAPS** (randomises local admin passwords to stop lateral movement). *Once an attacker has local Admin, there is virtually no software defence against pwdump/Cain/Ophcrack — the only real defence is preventing Admin in the first place.*

4.12 Dumping Cached Passwords

Even without the SAM, cached credentials are targets:

- **LSA Secrets** (**HKLM\SECURITY\Policy\Secrets**) hold plaintext service-account passwords, cached hashes of the last ten users, FTP/web passwords, **RAS dial-up** credentials, and computer-account passwords — encrypted on disk but **decrypted in memory** after login. Cain's "LSA Secrets Dumper" extracts them.
- **Previous Logon Cache (MSCASH/MSCASH2)**: when the DC is unreachable, Windows lets users log in with the **last ten** cached domain logons (encrypted/hashed). **CacheDump** reverses it; crack offline with `john --format=mscash hashes.txt` . (*Danger: a Domain Admin who recently logged in for maintenance leaves a crackable hash in the cache.*)

Countermeasures: set **CachedLogonsCount** from 10 to 0 (but then users can't log in when the DC is offline); Domain Admins must **never interactively log on / RDP to untrusted machines**.

4.13 Remote Control and Backdoors

After gaining Admin/SYSTEM, attackers install **backdoors** for persistent hidden access:

- **Netcat**: `nc -l -e cmd.exe -p 8080` makes the host serve a `cmd.exe` shell on port 8080; the **-d** flag runs it **stealth** (no console). Connect from anywhere with `telnet 192.168.11.2 8080` — **no Windows logon required**.
- **PSEXEC** (Sysinternals) runs over **SMB (139/445)** but **requires valid credentials** — ideal once admin creds are harvested: `psexec \\10.1.1.1 -u Administrator -p password -s cmd.exe` .
- **Metasploit** payloads spawn shells on stealthy listening ports.
- **GUI**: enable **RDP (TCP 3389, off by default)** or install **VNC** (edit the remote registry to start it stealthily at boot).

Countermeasures: `netstat -aon` to spot anomalous listening ports mapped to PIDs; monitor services and Registry Run keys; firewall/**egress filtering** for non-standard ports; AV/EDR; on certain compromise, **wipe & reinstall**.

4.14 Port Redirection

When firewalls block standard remote-control ports (e.g. Telnet 23), attackers **tunnel over an allowed port**. McAfee's **Fpipe** is a TCP port redirector: `fpipe -v -l 53 -r 23 192.168.234.37` makes a compromised "redirector" **listen on the allowed port 53** and forward to the target's blocked port 23, subverting the firewall.

4.15 Covering Tracks and Hiding Evidence

- **Disable auditing:** `auditpol /disable` (defenders restore with `/enable`). Native from Vista; in the Resource Kit for XP/NT/2000.
- **Clear the Event Log:** tools like **ELsave** or **Clearlogs.exe** wipe Security/System/Application logs.

Exam nugget: mass log clearing itself generates an uncleanable **Event ID 1102** ("Audit log was cleared") before reboot — the primary alarm for a SOC analyst.

- **Hide files:** `attrib +h filename` (trivially bypassed by showing hidden files).
- **Alternate Data Streams (ADS):** an NTFS feature (originally for Mac compatibility) that hides data/executables in a file's metadata **without changing its visible size**. `echo "1234567890" > original.txt:more.txt` hides text; `dir` shows only `original.txt`. Hide a binary with `cp nc.exe oso001.009:nc.exe` and run it via `start oso001.009:nc.exe`. Detect with **LADS** or Foundstone's **SFIND**; **permanently remove by copying the file to a FAT partition** (no ADS support) and back.
- **Rootkits (deepest hiding):** placed at three levels — **User-Mode (Ring 3)** API hooking (e.g. intercepting Task Manager to hide the malware name; Hacker Defender), **Kernel-Mode (Ring 0)** disguised as **.sys drivers** modifying kernel structures (**SSDT/IRP**; TDSS, Rustock) — sitting "below" the AV, and **Bootkits** in the **MBR/VBR** loading before the kernel. Detect with **GMER** or an **offline scan** (mount the disk from a clean OS). Prevention: **Driver Signature Enforcement**, **PatchGuard (KPP)**, **Secure Boot**. (*Writing Ring-0 rootkits is very hard — a small error causes a BSOD, revealing the attacker.*)

- **WHIPS (Windows Host IPS)**: a kernel module controlling critical system-call invocation to block deep modifications.
- Forward logs in real time to an **immutable SIEM** so local clearing doesn't erase evidence.

4.16 Incident Response — The Four Suspicious Areas

If cleaning rather than reinstalling, responders (working from **trusted USB tools**, ideally with the machine isolated) examine four vectors where malware must live:

1. **Suspicious files**: hunt known names (`nc.exe`), run AV, and use file-integrity monitoring (**Tripwire**).
2. **Suspicious registry**: scour **ASEP Run keys** — `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run` and user hives (e.g. `...\ORL\WINVNC3`, `Net Solutions\NetBus Server`); purge with **reg delete**.
3. **Suspicious processes**: watch **Task Manager / Process Explorer** for high-CPU or misparented processes; check the scheduler (`schtasks.exe`).
4. **Suspicious ports**: **netstat -aon** maps listening ports to PIDs, tracing rogue connections to their executable.

Best practice on Admin-level compromise: **wipe and reinstall** — with rootkits and ADS, you can never be certain all backdoors are gone.

(Limitation: if a kernel rootkit is installed, user-space IR tools are blind.)

4.17 Core Windows Security Features & Defense Architecture

Network & policy: **Windows Firewall** ("exception" model — inbound blocked by default); **Automated Updates / Security Center**; **Group Policy** (enterprise-wide config enforcement); **Microsoft Security Essentials** (free antimalware, built into Win8).

Memory mitigations (EMET & compiler): buffer security checks (**GS**) and **SafeSEH** (automatic); via **EMET: DEP** (marks memory non-executable, stopping injected-payload execution) and **ASLR** (randomises module memory locations, defeating predictable-address exploits).

Data security: EFS (per-file encryption — the file's symmetric key is encrypted with the user's public key; *note*: in Win2000/2003 the Local Administrator was the Default Recovery Agent — a big risk — fixed in XP+). **BitLocker** (full-volume, protects against offline attacks); **BitLocker To Go** (removable drives, Win7+).

- **Cold Boot attack**: BitLocker keys sit in volatile **DRAM**; an attacker freezes the chips with compressed air to preserve data, moves the RAM, and dumps the key before it fades. Countermeasure: require a **separate removable module (USB startup key)**.

System hardening: Least Privilege (log on as Limited User, use "**Run as**" to elevate). **Windows Resource Protection (WRP)** — successor to **WFP** (Vista+) — protects critical system files (`.dll` / `.sys`), folders, and registry keys via strict **ACLs**; **not even Administrator or SYSTEM has write access** — only the **TrustedInstaller** principal (the Windows Modules Installer, during updates) can overwrite them, blocking silent library replacement by user-mode rootkits.

WRP limitation: an admin can still use **takeown** to seize ownership and **icacls** to rewrite the ACLs, so WRP mainly stops *accidental*/third-party changes, not a determined admin/attacker. The real defence is **Least Privilege**.

Mandatory Integrity Control (MIC), UAC & Integrity Levels — derived from the Biba model, MIC assigns processes/objects **Integrity Levels: Low, Medium, High, System**. An admin login yields **two tokens** — a filtered **Medium** and a full **High**. Standard apps and the browser run at **Medium/Low** (Protected-Mode IE); an action needing system changes requests **High IL**, triggering **UAC** (the "Do you want to allow...?" prompt) to consent. This **confines exploits** — a compromised Low-IL browser can't write to the Windows folder or registry (drive-by download neutralised). Inspect with **icacls** and Sysinternals **Process Explorer**.

MIC/UAC limitations: Microsoft does **not** consider UAC a **security boundary** — advanced "**UAC bypass**" (e.g. DLL hijacking in auto-elevating folders) exists, and **alert fatigue** means a user clicking "Yes" defeats it. Set UAC to "**Always Notify**" and use **Standard User** accounts.

Windows Service Hardening: resource isolation, least-privilege services, per-service network restriction, and **Session 0 isolation** (separating background

services from the UI).

Best practices / CIS. The **Center for Internet Security (cisecurity.org)** provides free Microsoft configuration **benchmarks** and scoring tools. **Core security summary:** (1) minimise/disable unnecessary services (spooler, SMB); (2) always deploy the firewall and protect Internet-facing servers; (3) keep systems patched; (4) strictly limit interactive logon; (5) enforce config via Group Policy; (6) enforce physical security (offline boot, Cold Boot); (7) remember end-user apps are often more vulnerable than the OS; (8) stay informed and watch secondary products (e.g. SQL).

CHAPTER 6 — Cybercrime and Advanced Persistent Threats (APT)

Note: Chapter 5 (Unix Hacking) is not present in the source theory document, so this guide jumps from Windows to APT. Core Linux post-exploitation is nonetheless covered in §6.13 below.

6.1 What Defines an APT

An APT is defined by three characteristics:

- **Advanced** — sophisticated methodologies, often **zero-day or custom exploits** tailored to the target.
- **Persistent** — the attacker does not breach and leave; they **return repeatedly to maintain a foothold**, pursuing a long-term goal while staying **completely undetected**.
- **Threat** — the groups behind them are **highly organised, well-funded, and deeply motivated** (not amateurs).

6.2 APT vs Non-APT Attacks

Non-APT attacks hit "**targets of opportunity**" — scan the Internet, find a vulnerable system, and use a brief "**smash-and-grab**" to steal what they can quickly. APTs, by contrast, are **deliberately designed to steal massive sensitive data from a specific corporation over a prolonged period**. They do **not** want to destroy systems or interrupt operations — the primary objective is to **stay hidden** so the stream of stolen data keeps flowing back undetected.

6.3 Motives: Crime vs Espionage

- **Crime** — financially motivated: steal **PII, financial info, or corporate data** for fraud.
- **Espionage** — sponsored by rival industries or **nation-states**: steal **intellectual property / trade secrets** for competitive advantage.

Either way, the ultimate goal is to **gain and maintain access to targeted information**.

6.4 Intrusion Techniques and Evasion

Most APT campaigns begin with **spear-phishing** — highly targeted deceptive e-mails tricking a specific user into installing malware. Evasion techniques:

- **Cut-outs**: route attacks through **previously compromised computers** to hide the true location.
- **Dropper delivery services**: "**pay-per-install**" or leased malware campaigns to deliver payloads.

Other vectors: **SQL injection** to plant malware on websites, **infected USB sticks** dropped physically, **pre-infected hardware/software**, **social engineering** (impersonating authorised users), and occasionally **compromised insiders**.

6.5 The Six Phases of an APT Campaign

1. **Targeting** — collect information; test defences via vulnerability scanning, social engineering, spear-phishing.
2. **Access / Compromise** — gain a foothold, ascertain host info, quietly collect more credentials, use malware to obfuscate intent.
3. **Reconnaissance** — systematically enumerate surrounding networks and internal systems (e.g. **LDAP/MSRPC** queries).
4. **Lateral Movement** — use mapped data and stolen credentials (**Pass-the-Hash / Pass-the-Ticket**) to move horizontally to more valuable hosts.
5. **Data Collection & Exfiltration** — establish internal collection points, then exfiltrate packaged data, typically **via proxies**.
6. **Administration & Maintenance** — establish **backdoors** to maintain long-term access.

6.6 Detecting APTs and Forensics (overview)

Detection is multi-faceted: scrutinise **e-mail logs** for the initial vector; lateral movement leaves artifacts from **credential/identity misuse**; exfiltration traces sit across **firewall, IDS, DLP, application-history, and web-server logs**. Deep forensics examines **live file systems and RAM** plus full **hard-disk images** of suspected machines. The defensive stance is **defense-in-depth**, with **egress filtering** (monitoring outbound traffic — often neglected), a **SIEM** for log aggregation/correlation, and **FIM (File Integrity Monitoring)** to catch silently modified system files.

6.7 Historical APT Campaigns

Major campaigns: **Operation Aurora**, **Nitro**, **ShadyRAT**, **Lurid**, **Night Dragon**, **Stuxnet**, **DuQu**.

Operation Aurora (2009) targeted US tech/defense — **Google, Juniper, Adobe** and 29+ others, over up to six months. Vector: a spear-phishing link to a Taiwanese site with malicious **JavaScript** exploiting an **IE zero-day** undetected by AV; this dropped Trojan downloaders that installed a **backdoor RAT accessed over SSL**. Inside, attackers did reconnaissance, **compromised Active Directory credentials**, and moved laterally to IP-rich shares. Downloaders traced to Taiwan; C&C traced to two Chinese educational institutions. Google blamed China, but no definitive state-sponsorship proof — likewise **Night Dragon (2010)**, the **RSA Breach (2011)**, and **Shady RAT**; C&C nodes are hosted globally.

Anonymous (2011) — not a traditional APT but a loose collective aiming to **expose information publicly and disrupt services via DoS**, not for financial/competitive gain. Techniques: **SQLi, XSS, web-service exploits, social engineering (spear-phishing, help-desk impersonation)**. Targets included government agencies, **BART, Mastercard, Visa**. Their notorious **Sony PSN hack (Apr 2011)** exposed **~77 million accounts** (usernames, addresses, DOB, passwords, credit/debit details) and forced a **23-day** network outage.

Russian Business Network (RBN) — a St. Petersburg criminal syndicate running massive **botnets** for spam/phishing/malware, focused on **identity and financial theft**, and acting as a **crime provider** (platform rented to other criminals).

6.8 Case Study: GhostNet (2008–2010)

GhostNet targeted the **Dalai Lama** and the Tibetan Government-in-Exile (offices in India, London, NYC), using the RAT **GhostRAT**, whose capabilities included: **clearing SSDT hooks** (removing existing rootkits to dominate), full **file/process explorer**, **keystroke logging**, **remote screen control**, **webcam and microphone eavesdropping**, screen/mouse/keyboard disabling, dial-up profile cracking, remote binary download, a remote shell, and session management.

Lifecycle: a spear-phishing e-mail (from a blacklisted server, e.g. routed via `ObiWan@mailplanet.com`) → victim clicks → **GhostRAT installs and hides to survive reboot** → connects to **C&C** → attackers check the domain, create accounts, and hop laterally via **Terminal Server** (logged in Event Logs) → modify system files (detectable via a **diff on \System32**) → compress documents into **ZIP** for exfiltration → establish a **second netcat backdoor**, create an FTP account, and **schedule a daily log-cleaning job**. Investigators trace phishing origins with **Robtex** (reverse DNS/Whois/AS) and **PhishTank** (community phishing-URL database).

6.9 Indicators of Compromise & Persistence

Malware survives reboot via: **"Run" registry keys**, creating a **new service**, **hooking an existing service**, a **scheduled task**, disguising communications as valid traffic, or overwriting the MBR/BIOS.

6.10 The Order of Volatility (RFC 3227)

Acquire evidence from the most volatile (vanishes fastest) to the most stable, guided by **RFC 3227**:

1. **CPU registers & cache** (survive fractions of a second)
2. **RAM (volatile memory) + routing table / ARP cache** (vanish at shutdown; hold current system/network state)
3. **Page/swap file & hibernation file (`hiberfil.sys`)** (RAM extensions on disk)
4. **Running process information**
5. **Network data** (listening ports, active connections)
6. **System Registry** (+ temporary logs — overwritten by normal activity)
7. **Forensic bit-stream image of the disk(s)**
8. **Backup / remote archive media** (most static)

Golden rule: to preserve volatility, **isolate the machine from the network** (unplug Ethernet / disable the switch port) **but do NOT pull the power** — cutting power or a BSOD destroys the first items irrecoverably. Only RAM holds **cleartext passwords (from LSASS), decryption keys (BitLocker/TrueCrypt), live C&C connections, and fileless malware**, so preserving it is critical.

6.11 Memory and Process Forensics

Why RAM matters: modern malware evades disk-based defences via **process injection / process hollowing** — injecting code into a **legitimate process** (e.g. `svchost.exe`, `explorer.exe`) — and on-disk payloads are **packed/encrypted**. Because the **CPU cannot execute encrypted instructions**, the payload **must be decrypted and reside in cleartext in RAM** at execution time; a memory dump freezes that state, bypassing the attacker's crypto. Analysing RAM guarantees **unencrypted** data and recovers **fileless malware, injected DLLs, cleartext RAT commands/URLs/C&C IPs, credentials/NTLM hashes/Kerberos tickets, and BitLocker/ransomware keys**.

- **Acquisition:** **AccessData FTK Imager, Magnet RAM Capture / DumpIt** (fast, agentless, full crash dump without a BSOD) — run from a **clean USB** to minimise memory pollution; the PC must stay **powered on**.
- **Analysis:** the open-source **Volatility Framework** (released 2007, Black Hat DC) is the standard for inspecting processes, network connections, and extracting suspicious DLLs for string analysis; also examine `pagefile.sys / hiberfil.sys` with **HBGary FDPro** or **Mandiant Memoryze**.

Limitation: Volatility is not automated — it requires the exact OS "**Profile**" (build/patch) and deep knowledge of kernel structures (**EPROCESS**, offsets). And some malware watches for memory-reading APIs and **self-deletes or forces a BSOD** (anti-forensics) when a dump tool starts.

6.12 File and Process Capture Workflow

- **File metadata & network:** review the **MFT** for the **MACB timeline** (creation, modification, access, entry-modified) — beware **timestomping**; `netstat -aon` for connections ↔ PIDs; check the **Host file**

(`C:\Windows\System32\drivers\etc\hosts`) for malicious redirects (Pharming).

- **Ports & processes:** **CurrPorts** maps open ports to DLLs and remote IPs (an **unsigned DLL in `svchost.exe`** is a strong IoC); **Process Explorer** inspects a process's DLLs, in-memory strings, services, and `cmd.exe` executions.
- **Kernel & memory:** **Process Monitor** tracks process ↔ kernel interactions (thread creation, TCP lengths); **VMMMap** visualises a process's virtual/physical memory (image/heap/stack/private) and its strings for RAT indicators.
- **Network & config artifacts:** **DNS Cache** (`ipconfig /displaydns`) shows attempts to resolve C&C or infected hosts; `reg query` for suspicious **Run/RunOnce** keys; `at` for malicious scheduled tasks.
- **Logs & history:** pull Event logs with **psloglist** (new processes, users added to local Administrators); the **Prefetch directory** (`C:\windows\prefetch`) shows the **last 128 unique programs executed**.
- **Profiles & AV:** collect `ntuser.dat` (profiles), `index.dat` (requested URLs), `.rdp` files (servers hopped to), and **BMC files** (cached RDP bitmaps — view what the attacker saw); check **AV logs for attacker-created exclusions** (letting `netcat run`) and packing; use **Wireshark** for packet-level C&C traffic (e.g. HTTP payloads carrying encoded RAT commands).

*(These 8 checks — Netstat, MFT, Host file, CurrPorts, DNS cache, Registry Run keys, Event Logs, Prefetch — are the key subset of the recommended 22 forensic controls. Send everything to an **immutable central SIEM**. Limitation: a **kernel rootkit** makes these on-host tools "lie," hiding malicious processes/files.)*

6.13 Investigating a Linux APT Scenario

A common scenario: a Linux server running **Apache Tomcat with weak/default credentials** (lazily copied from an example config). Attackers exploit it via **Metasploit** through the Tomcat service, then `cat /etc/passwd | more` to map usernames (`root` , `daemon` , `bin` , `sys` , ...).

Escalation & persistence: find a user whose password is guessable (e.g. their surname) or crack the superuser password; then upload a **PHP backdoor** and create a **SUID root shell** (regaining root even after a password change). Convert the host into a **Metasploit pivot** to route deeper without heavy tools, running **Meterpreter in memory** (no disk writes) for near-zero trace.

Diagnosis: isolate via firewall, then check the **root history** and system logs for **sudo su -** (privilege escalation); review **Tomcat access logs** for **HTTP PUT** requests uploading files; inspect **.bash_history** (in each user's home; size set by **HISTFILESIZE** in **.bashrc**, while **HISTSIZE** is only the RAM buffer) for commands like **find -name passwd** or **telnet** to other hosts; check connections with **netstat -anlp** and **lsof -i -P** (lists listening services and established outbound C&C connections).

Caveat: an installed rootkit can make these standard programs lie and hide connections.

Where attackers hide Linux files: directories named **..** (dot-dot-space, hard to spot); drive slack space, **/dev**, **/tmp**, **/var/tmp**; and **RAM drives** (vanish on reboot) — **/dev/shm** is a default RAM drive, or create one with **mkdir -p /tmp/ram && mount -t ramfs -o size=512M ramfs /tmp/ram/**. Spot hidden RAM disks with **df -a**. Analyse suspicious binaries with **strings malware.exe** to extract referenced libraries (**mscorlib**, **kernel32**, **.NET v2.0.50727**).

6.14 Prominent Malware and Botnets

- **Poison Ivy** — a widely available RAT (source public), a staple in **Operation Aurora (2009)**, **RSA (2011)**, **Nitro (2011)**, frequently coupled with IE zero-days for silent install.
- **TDSS (TDL 1–4)** — a sophisticated botnet of **5M+ hosts** with advanced **rootkit** capabilities, encrypted files/comms, and many C&C servers; spawned **ZeroAccess** and **Purple Haze**. A prime example of "Malware-as-a-Service," rented out for **DDoS**, **click fraud**, and installing others' Trojans.

6.15 The 14-Step Anatomy of an APT Attack

1. **Spear-phishing e-mail**.
2. User clicks the link → opens an app and is redirected to a hidden address.
3. That "**Dropsite**" detects browser vulnerabilities and drops a **Trojan downloader**.
4. The downloader sends a **base64-encoded** instruction to a different dropsite to install the primary backdoor.
5. The backdoor installs in **c:\windows\system32** and registers in the **NETSVCS**

registry keys to survive reboot.

6. It uses a filename **only slightly different** from legitimate Windows files.
7. It establishes **SSL-encrypted C&C** communication.
8. The attacker interacts via **cut-outs** over SSL to mask identity/location.
9. Lists Computername/User Accounts; uses **Pass-the-Hash** to steal local and AD credentials.
10. Performs **service privilege escalation** for broad reconnaissance.
11. **Cracks stolen hashes offline.**
12. **Lateral movement** using legitimate admin tools to blend in — **RDP**, **SC.exe** (create remote services), **NET** commands (connect to shares).
13. On new high-value targets, installs **additional backdoors** and new egress points.
14. Packages stolen IP into **ZIP/RAR** archives **renamed with .GIF** extensions before exfiltration.

6.16 Detecting and Mitigating APTs

Defense-in-depth: strict **auditing of file-system changes**; **SMS alerts on administrative logins**; at the boundary, firewalls that **restrict inbound RDP, VNC, and CMD.EXE** access; internally, **AV, HIPS, and continuous file-integrity checking**; and relentless traffic monitoring via **NIDS/NIPS (Snort)** with all logs aggregated and correlated in a **SIEM**.

CHAPTER 7 — Remote Connectivity and VoIP Hacking

The threat landscape here (from *Hacking Exposed 7*) covers **Dial-up, PBX, Voicemail, VPN, and VoIP** attacks — the ways an attacker reaches a network *outside* the standard Internet-facing perimeter.

7.1 Dial-Up Hacking

Despite broadband, many companies keep **dial-up** connections to reach old servers, obscure devices, and **Industrial Control Systems (ICS)** — often forgotten and unmonitored. The process mirrors normal hacking: **footprint → scan → enumerate → exploit**.

Reconnaissance. Begins with **phone-number footprinting** — identifying blocks of numbers to load into an automated **wardialer**, harvested from phone directories, target websites, Internet registration databases, and manual dialing. *Countermeasures:* require passwords for account inquiries, sanitise public info, and educate employees.

Hardware & software. Attackers use many high-quality **modems** to scan fast, but face **logistical hurdles** (long-distance/international/connection charges) and **legal issues** (laws on identifying lines, recording calls, spoofing numbers). Software is judged on **scheduling, ease of setup, and accuracy** at detecting live lines. Tools: **WarVOX, TeleSweep, PhoneSweep.**

Brute-force scripting & Penetration Domains. Discovered lines are classified into **penetration domains** (the "homegrown way"), and brute-forced with terminal software/scripting — **ZOC, Procomm Plus, ASPECT.** The five domains dictate strategy:

- **Low-Hanging Fruit** — easily guessed/common passwords.
- **Single Authentication, Unlimited Attempts** — one auth factor, **no disconnect** after failures (infinite guessing).
- **Single Authentication, Limited Attempts** — one factor, **disconnects** after N failures.
- **Dual Authentication, Unlimited Attempts** — two factors (ID + password), but **no disconnect.**
- **Dual Authentication, Limited Attempts** — the most secure: two factors **and disconnect** after N failures.

Dial-up security (cyclical 12-step framework). Inventory all dial-up lines → consolidate to a **central modem bank** → keep it as an **untrusted connection off the internal network** → make analog lines hard to find → business lines disclose no identifying info → require **dial-back authentication** → physically secure telecom closets → monitor dial-up software logs → require **MFA** for all remote access → train the help desk on credential sensitivity → centralise provisioning → set firm policies → **loop back to step 1** for continuous inventory/monitoring.

7.2 PBX and Voicemail Hacking

PBX hacking takes the **exact same route as dial-up** (dial-up lines on PBXes are used for remote management, especially by vendors). Known **default**

credentials are the prize:

- **Octel Voice Network Login** — numeric password defaults to **9999**.
- **Williams/Northern Telecom** — user number + a four-digit numeric access code.
- **Meridian Links** — often **maint/maint**; **Rolm PhoneMail** — often **sysadmin/sysadmin**.
- A PBX protected by **RSA SecurID** is robust — "take a peek and leave."
Countermeasures: reduce modem uptime; deploy multiple authentication factors.

Voicemail brute-force (like dial-up) needs three things: (1) the **main phone number** for the voicemail system, (2) the **target mailbox number** (typically **3–5 digits**), and (3) an **educated guess of the PIN** (usually all numeric — a small, attackable keyspace). Tools: **Voicemail Box Hacker 3.0**, **VrACK 0.51** (good vs old systems), **ASPECT**. *Countermeasures:* **lockout** after failed attempts, and continuous logging/monitoring of voicemail connections.

7.3 VPN Hacking

VPNs replaced dial-up as the primary remote-access method but introduce new vectors.

Google hacking for VPN. The operator **filetype:pcf** finds exposed **PCF** (**Profile Configuration Files**) for the **Cisco VPN client**. The attacker downloads/imports the file, connects, and pivots; the PCF's stored **Group Password** is only weakly obfuscated and is **decoded offline with Cain** for password-reuse attacks. *Countermeasures:* user awareness, sanitise public sites, and **Google Alerts** to catch leaked documents. (*Fails if the org uses individual PKI certificates/IKEv2 or mandatory 2FA instead of group PSKs.*)

Probing IPsec VPN servers. Check whether **UDP 500** is reachable, then perform **IPsec identification + gateway fingerprinting** to determine the **IKE Phase 1 mode** and server hardware. Tools: **Nmap**, **NTA Monitor**, **IKEProbe**. Admins can do little to prevent the probe itself (though the probe alone compromises nothing).

Attacking IKE Aggressive Mode. The key flaw: **IKE Phase 1-Aggressive mode does not establish a secure channel**, exposing authentication material to eavesdropping. The attacker confirms aggressive-mode support with **IKEProbe**, initiates a connection, captures the auth messages with **IKECrack**

and **Cain**, and cracks them **offline**. *Countermeasures: discontinue Aggressive mode entirely and use token-based authentication. (The offline crack only succeeds against weak PSKs; it fails against Main mode or strong certificate/token auth.)*

Hacking Citrix (App Breakout). Citrix client-to-site VPNs often **publish single applications** (Word, IE) rather than a full desktop. The attack is an "**App Breakout**": abuse a published COTS app's legitimate OS interaction to spawn a system process (**explorer.exe**, **cmd.exe**, **PowerShell**). *Concrete mechanism*: open a published **Notepad**, choose **File** → **Save As...** to bring up the remote **Explorer** window, then via the address bar / right-click / Help menus, type and execute **cmd.exe** — yielding an interactive shell inside the corporate DMZ, authenticated and encrypted through the Citrix tunnel, running with the Citrix user's privileges. The ten popular breakout categories: **Help**, **MS Office**, **IE**, **Games/Calculator**, **Task Manager**, **Printing**, **Hyperlinks**, **Internet Access**, **EULAs/Text Editor**, and **"Save As / File-System Access."** *Countermeasures*: isolate Citrix in a **segmented/monitored DMZ** (never the trusted intranet), enforce **MFA**, and apply strict **application whitelisting (AppLocker/GPO)** that forbids **cmd.exe / PowerShell / Task Manager** and disables right-click. *(The attack depends 100% on misconfiguration — proper hardening makes breakout nearly impossible.)*

7.4 VoIP Attacks

VoIP transports voice over IP using signaling protocols **H.323** and **SIP**.

SIP scanning & TFTP pillaging. Phase 1 is **SIP scanning** to discover SIP proxies/devices with **SiVuS** and **SIPVicious** (**svmap**). Attackers also **pillage TFTP**: SIP phones fetch their **config files from a TFTP server at boot**, and since **TFTP has no authentication**, the attacker locates it with **Nmap (UDP 69)** and brute-forces the exact config filename (e.g. **mac_address.cfg**) to extract **cleartext SIP credentials, server IPs, and admin passwords**.

Countermeasure: **network segmentation** and restricting TFTP access (recall TFTP is "blind" — no **DIR / LS**, so the filename must be guessed).

User enumeration. SIP is **human-readable**, so attackers enumerate extensions by sending mass **INVITE/REGISTER** requests and reading responses:

- **200 OK / 401 Unauthorized** → the extension **exists** (server asks for a password).

- **404 Not Found / 403 Forbidden** → the extension does not exist. Tools: **SIPVicious** `svwar.py`, **SIPScan**, **Sipsak**; targets include **Cisco Directory Services**. A compromised extension enables **toll fraud**, **eavesdropping/call hijacking**, and **Vishing/Caller-ID spoofing**. *Countermeasures*: segment Voice/Data VLANs, deploy IDS/IPS, and **return identical generic error codes** (always **404**) so enumeration can't distinguish valid extensions.

Interception (eavesdropping). VoIP has two streams: **signaling (SIP, or SKINNY/UNISTim)** and the **media stream (RTP — Real-time Transport Protocol)**. The attacker gets **MITM via ARP spoofing** (`dsniff`, `arp-sk`), sniffs the **RTP UDP packets** (`tcpdump`, **Wireshark**), identifies the **voice codec** via the **Payload Type / Media Format** field (e.g. **G.711, G.729**), **reorders packets by timestamp** (to compensate for jitter), and converts the stream to a playable file (`.wav`) with **vomit**, **scapy**. All-in-one/offline tools: **UCSniff**, **SIPdump**, **SIPcrack**, and **Wireshark** (RTP & Cisco SKINNY dissectors — its **Telephony → VoIP Calls** feature literally lets you press **Play**).

Countermeasures: the definitive fix is **SRTP** (encrypts the audio payload with **AES** — sniffing yields only noise) plus **TLS** to hide SIP signaling, alongside an isolated **Voice VLAN**. (*Requires logical proximity to the target subnet; fails entirely against SRTP+TLS.*)

Denial of Service. Achieved by flooding the target with **fake call-setup signaling** — abusing the **SIP INVITE** command (usually on **UDP/TCP 5060**) — or by flooding an IP phone with unwanted **unicast/multicast** traffic. Tools: **Inviteflood**, **hack_library**. This extracts no data; the impact is pure DoS (the PBX collapses; employees can't call). *Countermeasures*: strict **Voice/Data VLAN segmentation**, **authentication + encryption for all SIP**, and robust **IDS/IPS** or a **Session Border Controller (SBC)** — which instantly detects the anomalous SIP-setup spike and bans the source IP.

7.5 Summary — Remote-Access Security Tips

Password policy matters even more remotely — strongly prefer **two-factor authentication**. Develop comprehensive **provisioning policies** for all remote access and **eliminate unsanctioned remote-control software**. Remember that **PBXes, fax servers, and voicemail systems** are attack vectors alongside modems. Educate support staff and users, be **skeptical of vendor security claims**, and enforce a strict use policy backed by **regular audit compliance**.

CHAPTER 8 — Wireless Hacking

8.1 Aircrack-ng and the WEP Case Study

Aircrack-ng is the core WiFi-security suite — command-line (scriptable), cross-platform, for **monitoring** (packet capture/export), **active attacks** (replay, deauthentication, fake APs via **packet injection**), injection testing, and **cracking WEP and WPA-PSK (WPA 1 & 2)** keys.

Case study (retail WEP POS): from the parking lot, with a WiFi card, directional antenna, and Aircrack-ng, the attacker sets the NIC to **monitor mode**, runs **airodump-ng** to sniff 802.11 frames for the **SSID, AP MAC, and WEP IVs**, uses **aireplay-ng** to spoof a client and **replay an ARP packet** (forcing the AP to generate masses of IVs), then runs **aircrack-ng** to mathematically recover the key from the capture — finally joining the network via DHCP.

8.2 802.11 Fundamentals

Wireless uses unlicensed **ISM bands**. The **2.4 GHz** band (802.11b/g/n) has **channels 1–14**, of which only **1, 6, 11** are **non-overlapping**. The **5 GHz** band (802.11a/n) spans **channels 36–165**, all non-overlapping. Two structural modes: **Infrastructure** (a central **Access Point** relays all traffic — most common) and **Ad Hoc** (peer-to-peer, like an Ethernet crossover, no AP).

8.3 Session Establishment: Probe, Authenticate, Associate

A client sends a **probe request** for the SSID it wants (repeated on every channel), awaiting a **probe response**; a **broadcast probe** (no SSID) asks any AP to announce itself (APs can be told to ignore these). Then an **authentication request: open authentication** accepts any client without cryptographic checks; **shared-key authentication** exists but is almost exclusively WEP and rarely used. **WPA does not affect this initial authentication phase** — its encryption engages later. Finally the **association phase** (association request → response) links the device to the network.

8.4 Basic (Flawed) Defenses

Admins deploy weak defenses like **MAC filtering** and **"hidden" networks** (SSID omitted from beacons). Counter-intuitively, **announcing the SSID is more secure**: Windows Vista+ looks for beacons before connecting, and a hidden

network forces the *client* to continuously transmit **probe requests containing the SSID**, opening it to **AP-impersonation** attacks. Research shows smartphone probe requests leak known-AP names **in the clear**, which can **de-anonymise individuals** and profile crowds.

8.5 Encryption Standards: WEP, WPA, WPA2/3

The **802.11i** standard dictates modern crypto. Original **WPA** implements only part of 802.11i, using **TKIP**; **WPA2/WPA3** implement the full standard (**TKIP** for backward compatibility + **AES**). The three options:

- **WEP** — **RC4** cipher, fundamentally flawed, obsolete.
- **TKIP** — a software replacement for WEP (still RC4) that runs on old hardware; no major vulnerabilities currently known.
- **AES** — most secure: used in **WPA2 via CCMP** (Cipher Block Chaining MAC Protocol) and in **WPA3 via GCM** (Galois/Counter Mode, SHA-384 HMAC).

8.6 WPA Personal vs Enterprise

- **WPA Personal (WPA-PSK)**: a single **Pre-Shared Key** for all users, stored on every client; changing it means manually updating all clients + AP — no individual management.
- **WPA-Enterprise: 802.1x** + a backend **RADIUS** server + an **EAP** framework (EAP-TTLS, PEAP, EAP-FAST). Users present **individual credentials** and **never handle the encryption keys**, so an attacker **cannot extract the network key from a compromised client** — granular, per-user access control.

8.7 The Four-Way Handshake

Both WPA-PSK and Enterprise rely on the **Four-Way Handshake**, which establishes the **PTK (Pairwise Transient Key** — secures unicast between AP and one client) and the **GTK (Group Temporal Key** — secures multicast/broadcast). Steps: AP sends a random **ANonce** → client sends its **SNonce** → the AP computes the **PTK** from the nonces and sends an encrypted message → the client decrypts it with the derived PTK, confirming both share the key. The verification code inside the handshake is a **MIC** — an **HMAC-SHA1** that uses the network **SSID** and **MAC** addresses as **cryptographic salt**, i.e. mathematical proof of the password.

8.8 Equipment: Hardware, OS, Antennas

Manufacturer drivers limit NIC control, so **most off-the-shelf NICs can't do packet injection or raw monitoring**. Recommended cards are chosen by chipset — e.g. **Ubiquiti SRC (Atheros)** or **Alfa AWUS050NH (Ralink RT2770F)** — both USB, dual-band 802.11a/b/g/n, external-antenna capable. **Windows** has easy drivers but weak native tools (unless buying **AirPcap/OmniPeek**); **Linux** has superior open-source tools — hence **Kali Linux** (drivers pre-installed; run as VM with USB passthrough, bare metal, USB, or LiveCD).

Antennas: Omnidirectional (all directions, broad coverage) vs **Directional** (focused, longer range). Common directional types: **Cantenna** (home-built ~12 dBi in a can), **Yagi** (multi-element, highly directional — e.g. a 16 dBi 900 MHz Yagi reaching 20+ km), **Panel/Patch** (flat, for corridors), and **Biquad Dish** (biquad element + satellite dish for extreme distance).

8.9 Discovery, Monitoring, and Wardriving

Mapping networks from a moving vehicle is **wardriving** (often with **GPS** to geolocate APs). Tools read **802.11 management frames** (probes/beacons); because **source/destination MACs are always unencrypted**, client ↔ AP associations are easily mapped. Two methodologies:

- **Active discovery** — send broadcast probes and record responses (**NetStumbler**, **Windows**). **Misses hidden networks** that ignore broadcast probes.
- **Passive discovery** — put the NIC in **monitor mode** to silently record every AP MAC seen, transmitting nothing (**Kismet**, **Vistumbler** — the latter maps auth/encryption via GPS). Superior technique.

Wardriving data is crowdsourced in **WiGLE** (16.5M+ mapped points). Note the legal risk: **Google Street View** faced wiretapping lawsuits after accidentally sniffing/storing unencrypted payload data from open networks.

8.10 Sniffing and De-authentication DoS

Sniffing is trivial on unencrypted networks; the attacker's NIC must be in **monitor mode** (**Kismet**/ **airodump-ng** on Linux; **Wireshark** on **en0** on macOS reads beacon frames and the cleartext SSID). To reveal a **hidden SSID**, the attacker launches a **De-authentication DoS**: since **802.11 management frames**

are unauthenticated, `aireplay-ng` spoofs a deauth frame appearing to come from the AP (typically **64 deauth frames to the AP** spoofing the client and **64 to the client** spoofing the AP). The kicked client auto-reconnects, **broadcasting its Probe Request with the hidden SSID in clear text**, which the attacker captures.

8.11 Bypassing MAC Filtering (MAC Spoofing)

MAC filtering (whitelisting approved MACs) is high-effort, low-security: since client MACs travel **in the clear**, the attacker **sniffs an approved MAC and spoofs it**. Tools — Windows: `bwmachak` (old Orinoco), `SMAC 2.0` (KLC Consulting GUI), or natively via **Device Manager → Advanced → "Network Address."** Linux: `macchanger` or `ifconfig <iface> hw ether .`

Countermeasures: don't rely on MAC filtering as a primary defense; use strong encryption (WPA2/WPA3, ideally Enterprise); deploy a **WIPS** to detect spoofed/duplicate MACs. (*Precondition: at least one active legitimate client to sniff a valid MAC.*)

8.12 Exploiting WEP

WEP (RC4) is catastrophically flawed. Pure brute force would take weeks even for 40-bit keys, but WEP is defeated via its **Initialization Vectors (IVs)**.

Mechanism: the sender concatenates the WEP key with a pseudo-random **IV**, runs it through **RC4** to make a keystream, and **XORs** the plaintext to get ciphertext; the **IV is appended to the header in clear text**. The receiver uses the cleartext IV + shared key to recreate the keystream and XOR back. **The flaw:** IVs are sent in the clear and the **24-bit IV space is too small → IV reuse**. Duplicate IVs across frames let the attacker compare ciphertexts, deduce the RC4 keystream, and reverse-engineer the key (the historic **FMS** and **PTW** statistical attacks).

Two exploit paths:

1. **Passive:** `airodump-ng` captures data frames until enough IVs collect — roughly **60,000 unique IVs** to reliably crack a **104-bit** key (statistical PTW-class attacks succeed on the order of **40,000–85,000** captured packets) — then `aircrack-ng` statistically deduces the key.
2. **ARP Replay + Fake Authentication (active):** the attacker spoofs a client MAC, does a **fake open authentication** (no payload), captures a broadcast **ARP request**, and continuously replays it (`aireplay-ng -3`); each AP

reply is a **new packet with a new IV**, collecting 60,000+ IVs in **under 5 minutes** for an instant crack. Historic tools: **AirSnort**, **WLAN-Tools**, **DWEPCrack**, **WEPAAttack**. *(If no client is present to source an ARP, the attacker falls back to the harder **Fragmentation Attack** or **KoreK ChopChop**.)*

8.13 Attacking WPA/WPA2 Pre-Shared Keys (PSK)

WPA/WPA2 had no major structural weakness until 2017, but the **PSK** is vulnerable to **human error** (weak, shared password). The attacker does **not** crack AES/the algorithm — they target the **Four-Way Handshake**: monitor and wait for a client, or force a **deauth** (**aireplay-ng -0**) to make it reconnect, then **capture the handshake**. Cracking is entirely **offline**: the **PSK (8–63 chars)** is hashed **4,096 times** together with the **SSID** to derive keys — computationally heavy.

Tools: **aircrack-ng** (dictionary vs the PCAP), **coWPAtty** (precomputed **rainbow tables** — but because the **SSID is the salt**, tables are per-SSID; e.g. a 40 GB set for the top 1,000 SSIDs from WiGLE), **Pyrit** (offloads hashing to **GPUs**; Hashcat similarly). *Countermeasures*: a **complex, random PSK** + a **unique SSID** (defeats precomputed tables) + **disable WPS** (separately brute-forceable via its PIN). Fundamental weakness: the shared key means a network is **only as secure as its least careful user**. *(Precondition: at least one connected client to deauth/capture; a strong >20-char PSK makes the offline crack mathematically infeasible.)*

8.14 Attacking WPA-Enterprise and EAP

Attacking Enterprise means attacking the specific **EAP type**. First identify it: capture the **EAP handshake** in **Wireshark**, which dissects the EAP type (PEAP, Identity) — and note that early EAP often sends the **username in clear text**.

LEAP (the cautionary tale). Cisco's proprietary **LEAP** (2000) was a WEP band-aid, used by **46% of IT execs by 2004**, but is **fundamentally broken** — it relies solely on **MS-CHAPv2**, which **doesn't salt its NT hashes**, uses a weak **2-byte DES key**, and sends **usernames in clear text**. A 4 GB rainbow table cracks it in minutes. Attackers use **Asleap** (grabs/decrypts LEAP passwords) with **Air-Jack** (forcibly deauth users so they re-authenticate). Cisco's

defense was ≥ 10 -char complex passwords, but users rarely comply. In **2012** **Microsoft** warned to stop using **PPTP/MS-CHAPv2** after **CloudCracker** broke any MS-CHAPv2 in **<24 h** for **\$200**, recommending PEAP, L2TP/IPsec, or SSTP.

EAP-TTLS and PEAP (the TLS tunnel). These establish an encrypted **TLS tunnel** between client and RADIUS server (the AP just relays it, blind to contents); a weaker **inner** protocol (MS-CHAPv2, EAP-GTC) then validates credentials. The **outer TLS cannot currently be decrypted** — but the network falls to **AP impersonation (MITM)** if a client **doesn't validate the RADIUS server's certificate**. Using **hostapd** (rogue AP) + **FreeRADIUS-WPE** (Wireless Pwnage Edition), the attacker terminates the TLS tunnel, the misconfigured client connects and **passes its inner-auth credentials into the attacker's logs** (often forcing a **downgrade to MS-CHAPv2**), then **Asleap** cracks them offline.

Definitive countermeasure: enforce "**Validate server certificate**" on every client (specifying the trusted Root CA that signs the RADIUS cert). The strongest architecture is **EAP-TLS**, which requires a **client-side certificate** too — mutual authentication makes AP impersonation mathematically impossible.

8.15 Wireless Defense Summary

- **WEP** — fundamentally broken; passive sniffing + active ARP replay crack it in ~5 minutes. **Never use.**
- **WPA-PSK** — one shared key risks all users; vulnerable to **offline brute-force** of a captured handshake. Requires a highly complex passphrase.
- **WPA-Enterprise (LEAP)** — relies on flawed MS-CHAPv2; fast offline brute-force. **Do not use.**
- **WPA-Enterprise (EAP-TTLS/PEAP)** — highly secure via the TLS tunnel, but **clients must validate the server certificate** to stop MITM impersonation (EAP-TLS with client certs is stronger still).

CHAPTER 9 — Hacking Hardware

Hardware security splits into three core domains: **gaining physical access to facilities**, **hacking specific hardware devices**, and **reverse-engineering**

hardware components.

9.1 Physical Access: Mechanical Locks and Bumping

A mechanical lock's cylinder holds **key pins** (red in diagrams) and **driver pins** (blue); the plug only turns when the gaps between them align exactly at the **shear line** (yellow). **Bumping** exploits this: a **bump key** (e.g. AVVEL) is cut so every key pin drops to its lowest point; the attacker inserts it and **strikes it sharply** (e.g. with a screwdriver), and the kinetic shock makes the key pins **jump momentarily past the shear line** — in that instant the attacker turns the plug and it opens.

Impact: bumping is as fast as a real key, **leaves no evidence**, and **doesn't damage the lock**. It opens doors, server racks, PC cases, and laptop cable locks. At **DefCon 2007** researchers showed even high-security **Medeco M3** locks ($\approx 70\%$ of the market; used at the White House, Pentagon, embassies) could be bumped and picked.

Countermeasures: don't trust vendor "bump-proof" claims; use **layered physical security / two-factor** — **PIN** keypads, biometric scanners, or a human guard.

9.2 Cloning Electronic Access Cards

Magnetic stripe cards

Built on standard **ISO formats (7810/7811/7813)** with **three data tracks** (ID/serial/name/address) and the fatal flaw of **clear encoding** (no encryption at rest). *Track 1 "Format B" structure: Start Sentinel % → Format Code B → PAN (≤19 chars, matching the printed number) → Field Separator ^ → Name (2–26) → ^ → Expiration YYMM*. Cloning tools are cheap: an **MSR206** USB reader/writer (~\$35) plus software like **MAKStripe Explorer**. Two-step process: **Read** (swipe several same-type cards to see which bits change per user and deduce the checksum algorithm), then **Write** (recompute a **valid checksum** for the fabricated data and write it to a blank stripe). *(If a checksum byte guards integrity, the attacker cannot just change data — they must deduce and recalculate the checksum formula first.)*

RFID proximity cards

Transmit via radio at **135 kHz or 13.56 MHz** (now mandatory in passports), readable at a distance; early cards left transmission **unencrypted** to cut cost. The cautionary tale is the **MiFare Classic** (the most widely deployed contactless smartcard), which "rolled its own crypto" — a proprietary **48-bit stream cipher CRYPTO1**. It was partially reverse-engineered (Plötz & Nohl, Dec 2007) and **fully broken** (Radboud University, Mar 2008) using a **Proxmark** instrument + open-source **libnfc**, enabling cloning of the Dutch **OV-Chipkaart**. (NXP's injunction to block publication was **denied** — informing society of chip vulnerabilities outweighs it.) The **Boston MBTA** MiFare system fell too: MIT students (under Ron Rivest) simply **sniffed the challenge/response pairs** at the turnstile. Key-recovery uses **known-plaintext** attacks on FPGA rigs (e.g. **KwickBreak FPGA Brute-Forcer** on an Opal Kelly XEM3001, cracking Crypto-1/DES). Interception/clone hardware: **Proxmark3** (on-board FPGA, real-time decode) or a **USRP** (capture and blindly **replay** raw radio).

Countermeasure: move to a full **cryptographic challenge-response** — the **private key stays on the card**; energised by the reader, the card computes a response to a challenge that the reader validates (defeating clone/replay).

9.3 Hacking Devices: Locked Hard Disks (ATA Security)

Drives use **ATA** interfaces — older **PATA/IDE** (wide ribbon) vs modern **SATA** (thin, faster). **ATA Security** (on nearly every drive since 2000) requires a **BIOS password** for hardware-level disk access — but it **only blocks access; it does not encrypt the data**. Desktop BIOSes are often unaware of it, enabling an **"ATA Password Virus"** that maliciously turns on ATA security to **ransom/brick** a drive (no standard BIOS command resolves it).

Hot-Swap bypass: ATA security merely tells the controller not to respond until the BIOS supplies the password at boot. The attacker boots a computer with an **identical but unlocked** drive; the BIOS unlocks it and begins loading an OS (e.g. Linux Live); then, **with the PC powered on**, physically **hot-swaps** the SATA/IDE cables onto the locked victim drive. Since the BIOS **already passed validation**, the running OS sends read commands directly to the swapped controller, **bypassing the lock** — then clones bit-for-bit (e.g. `dd`).

- *Preconditions:* physical internal access; **same drive family/model** (so the motherboard controller doesn't crash); motherboard tolerating **SATA hot-plug**.

- *Risk:* hot-unplugging power/data live can cause **voltage spikes that fry the drive or motherboard.**

Forensic tooling: the **Vogon "Password Cracker Pod"** strips/changes the ATA password via GUI, images the disk, then **restores the original password** so the suspect never knows — by directly accessing the drive's user-inaccessible **"service area"** (firmware/geometry region).

Countermeasure: ATA security is fundamentally flawed — abandon it for **Full Disk Encryption (FDE): BitLocker, TrueCrypt, SecurStar, or LUKS.** (An FDE-protected drive hot-swapped still yields only encrypted, unreadable data.)

9.4 Hacking Devices: U3 USB Flash Drives

U3 smart drives (e.g. SanDisk Cruzer) carry data + preloaded apps like a "tiny laptop," using a **dual-partition** architecture: a normal **Removable Disk** plus a hidden **virtual CD-ROM ("U3")** whose software leverages Windows **AutoRun** to auto-launch the **U3 Launchpad**. Attackers overwrite that hidden partition with **Universal_Customizer.exe**, writing a custom ISO that edits **Autorun.ini** to silently launch **"Hak9's PocketKnife"** instead. On insertion, PocketKnife runs in the background to **kill antivirus, disable the Windows Firewall, slurp user files**, and — most dangerously — run **FGDUMP/PWDUMP** to extract the local **SAM password hashes**, plus dump caches, URL history, **Wi-Fi hex passwords**, external IPs, product keys, and cleartext passwords from Firefox/IE/**LSA Secrets**/mail/MSN.

Defenses: the devastation (culminating in **Stuxnet**) drove change — the **US Military banned USB thumb drives/CDs in Nov 2008**. Mitigations: block USB via **Group Policy**, disable **AutoRun** (Microsoft pushed a global fix Feb 2011), hold **SHIFT** while inserting to override AutoRun, or physically **glue ports shut**; the robust permanent fix is **IEEE 1667** (cryptographic authentication of transient storage, native since **Windows 7**).

9.5 Exploiting Default Configurations

Often no sophistication is needed — just **manufacturer defaults**. The **ASUS Eee PC 701** (Xandros Linux) shipped with **Samba enabled** in a vulnerable version, easily rooted via **Metasploit**. Routers/appliances ship with unchanged default passwords — e.g. **3COM CellPlex 7000** (**root / admin / tech** , passwords from **PASSWORD** to blank), **3COM Switch 3300XM/LANplex 2500**

(blank/ tech). Real-world consequence: in **Sept 2008, Foster & Eske** used embedded **ATM default admin passwords** to enter operator menus and reprogram machines to dispense **\$20 bills as \$1 bills**.

Bluetooth eavesdropping: encryption is often **off by default** and the pairing PIN is almost universally **0000** . Bluetooth uses **80 channels in the 2.4 GHz ISM band**, so attackers do **spectrum analysis** to intercept — eavesdropping headset audio, initiating unauthorized calls, transferring data, or **stealing contact lists** for social engineering. Tool: **Project Ubertooth** (Ubertooth One) for sniffing/playback of Bluetooth frames. *Countermeasures: enable encryption, change the default PIN, keep the device non-discoverable, disable Bluetooth when unused, update firmware.*

9.6 Reverse Engineering Hardware

Mapping the PCB & ICs. Identify the **IC chips** and Google their **datasheets** for packaging and pin diagrams; locate external interfaces (**HDMI, USB, JTAG**) and key data pins. Since multilayer boards defeat visual tracing, use a **digital multimeter in "toning"/continuity mode** — probe and listen for the beep of a closed circuit to map the internal **data bus**.

Bus sniffing. Chip-to-chip communication is usually **unprotected**, exposing it to interception/MITM/replay. A **Logic Analyzer** records the raw electrical signals; advanced ones have **software decoders** for **I²C, SPI, Serial**. If the bus is encrypted (e.g. DRM like **HDMI-HDCP**), the captured stream needs cryptographic analysis.

Sniffing wireless interfaces. Do a **Layer 2** attack: find the device's **FCC ID**, search the **FCC database** for exact **radio frequencies** and sometimes schematics, then capture with **SDR (WinRadio, USRP)** and **decode the symbols** to the lowest-level bits.

Firmware reversing (the ultimate prize — it holds **hardcoded default passwords, hidden admin/debug ports, backdoors, and crypto parameters like AES keys**). Extract physically with **EEPROM programmers**, or via hardware debuggers — **MPLAB IDE**, an **In-Circuit Emulator (ICE)** (an attacker might obtain the right ICE by contacting the vendor under false pretext), or by finding a **JTAG** port (whose pinout takes effort — "one size does not fit all"). Analyse the binary with **Hex editors (010 Editor)**, **strings** , and **disassemblers (IDA Pro)**.

Firmware mounting: rather than only opening the image in a hex editor/IDA Pro, the attacker can **mount the whole firmware filesystem image on Linux and browse it like a folder** with the exact sequence:

```
mount -o loop -t cramfs /tmp/cramfs
```

Then hunt for secrets directly, e.g. `find /tmp/cramfs -name *.key` — surfacing hardcoded **certificates and private keys** that let the attacker **forge entire SSL sessions**.

Countermeasures: encase sensitive chips in **epoxy resin (potting)** to block pin access; **disable or physically burn the JTAG pins** on production devices; **encrypt the firmware** in hardware; and never ship cleartext passwords/keys in public firmware. *(Limitations for the attacker: needs specialised, expensive gear, and many techniques are destructive or risk permanently "bricking" the device.)*

CHAPTER 11 — Mobile Hacking (Android & iOS)

*(Android dominates smartphones (~74%) and iOS leads tablets (~55%); the key security consequence of Android's lead is severe **platform fragmentation** — huge numbers of devices run outdated, unpatched OS versions.)*

11.1 Android Architecture

Android's core is an **ARM cross-compiled Linux kernel**. Above it sit native libraries (2D/3D graphics via OpenGL|ES/SGL, GPS) and an integrated **SQLite** engine that stores app data **without default encryption**. The **Android Runtime** historically used the **Dalvik VM**, which runs **.dex** files (compiled from Java) with **each app in its own isolated VM**; since Android 5.0 (Lollipop), **ART** replaced Dalvik for performance. On top is the **Application Framework** (Activity/Telephony/Package Managers, View System) and the apps. Its Linux base allows installing standard tools (**Nmap, tcpdump**) directly on the device.

11.2 Android Security Mechanisms

- **Sandbox** — enforced at the **kernel level**: each app runs in a **separate**

process with a unique **User ID**, preventing apps from interacting directly.

- **File-system security** — the **system partition is read-only** (unless rooted); one app can't modify another's files; from **Android 3.0** the file system is **AES-128 encrypted**.
- **Memory security** — **ASLR** and the **NX (No-eXecute)** bit prevent buffer-overflow execution.
- **Protected APIs** — users must explicitly grant permissions (fine location/GPS, contacts, full network, hardware, system tools, USB storage, account management).
- **Code signing** — every app must be signed, **but certificates can be self-signed** (no trusted-CA validation required) — a critical flaw exploited by repackaging.

Android malware has grown exponentially (from ~936 packages in Q3 2011 to millions of signatures), making mobile AV (e.g. **LookOut**) advisable.

11.3 The Four User Defenses (Android ≥ 3.0)

1. **Device Lock** — 4-digit PIN (weak), complex password, or **Pattern Lock** (vulnerable to **smudge attacks** — reading finger grease on the glass).
2. **Unknown Sources + Google Bouncer** — disabling "Unknown Sources" blocks sideloaded/URL-sourced malware; server-side, **Google Bouncer** analyses and removes malicious apps from the Play Store.
3. **Security Software** — beyond scanning: **Remote Wipe**, **GPS tracking** for theft, and blocking premium-rate call/SMS fraud.
4. **Full Disk Encryption (FDE)** — encrypts the whole data partition (Android 3.0+, native from 4.0), preventing hardware data extraction if lost (needs a strong boot PIN to be truly effective).

11.4 Development and Debugging Tools

The **SDK** / **Android Studio** include the **Android Emulator**. The **Android Debug Bridge (ADB)** communicates with a device over **TCP 5555–5585**; core commands: **pull** / **push** (files), **install**, **logcat** (logs), **forward** (redirect connections), **shell** (remote shell). The old **DDMS** (in Eclipse) was deprecated (removed in Android Studio 3.2), replaced by the **Android Profiler** / **Device Explorer** / **Debugger**.

11.5 Rooting (Hacking Your Own Android)

Rooting is a **privilege-escalation attack** exploiting kernel/LPE flaws or an unlocked bootloader to gain root — Android's equivalent of iOS jailbreaking. Risks: a failed root can **brick** the phone, and it **compromises OS security** (enabling malware). Rooting installs **SuperUser.apk** to gate `su` access. Modern rooting uses **Magisk** (unlock bootloader via `adb`, install the Magisk app, modify **boot.img**); historical tools were **SuperOneClick**, **Z4Root**, **GingerBreak** (with **Burrito Root** for the Kindle Fire). Rooted apps include ROM Manager, Market Enabler (spoof carrier/geo), ConnectBot (SSH), ES File Manager (edit system files), SetCPU/Juice Defender.

Cross-compiler: phones use **ARM**, PCs use x86/x64 — so an exploit compiled on a laptop won't run on the phone. Attackers use the **Android NDK** as a cross-compiler to build native **ARM ELF** binaries the phone's kernel can run, invisible to the Java layer. Precompiled tools: **BusyBox** (missing UNIX commands: `tar`, `dd`, `wget`), **tcpdump** (PCAP), **Nmap**, **Ncat**.

Detection/countermeasures: Google's **SafetyNet / Play Integrity API** cryptographically attest whether the OS is tampered, blocking sensitive apps on rooted devices; enterprises enforce **MDM**. *Limitations:* unlocking a modern bootloader usually **requires human interaction**, **triggers a factory reset** (wiping victim data), and trips hardware security flags (e.g. **Samsung Knox e-fuse**).

11.6 Trojan Apps and Repackaging

Malicious code is easily inserted into legitimate **APK** files disguised with real icons/names. An APK opens with **7-zip**: **Manifest.xml** defines components/permissions, **Classes.dex** holds the compiled Dalvik code. Exploitable entry points include **Broadcast Receivers** (silently run code on system intents like an incoming SMS) and **Services** (hidden background, no GUI). The **repackaging** process: disassemble the DEX to **Smali** (raw Dalvik bytecode), decode the manifest, embed malicious code, reassemble the DEX, re-encode the manifest, and **re-sign with a self-signed cert** (using **SignApk**). The main tool is **apktool**. *PoC:* editing Netflix's `strings.xml` to show "Hacking Exposed 7" on a connection failure.

11.7 Hacking Other Users' Android Devices

Fragmentation leaves unpatched targets highly vulnerable.

- **Remote Shell via WebKit** — Android's browser engine mishandled **floating-point data types** (patched in Android 2.2); a malicious HTML page triggered a silent **drive-by download** returning a **non-root remote shell**. To escalate, attackers ran **exploid** or **Rage Against The Cage (RATC)**. (*RATC mechanism: it abuses the **adb** daemon by spawning child processes in an infinite loop until **PIDs are exhausted**; when adb crashes and restarts, it fails to drop privileges for lack of resources and accidentally keeps running as root.*)
- **Data-stealing vulnerability** — a malicious link → PHP with embedded JavaScript extracts data from the device/SD card **without root**. *Countermeasures:* update (via a custom ROM like **CyanogenMod** if the carrier blocks updates), AV, disable JavaScript, unmount the SD card, or use a third-party browser (Firefox/Opera).
- **Remote Shell with Zero Permissions** — demonstrated by **Thomas Cannon** (worked even on Android 4.0/ICS): bypasses individual permissions creatively — **REBOOT** via a Toast-flood DoS, **INTERNET** via an invisible browser, a **BROWSABLE** custom-URI receiver to secretly receive data, and heavy abuse of **READ_LOG**.
- **Capability Leaks** — stock software exposes permissions to untrusted apps: **explicit** (an untrusted app invokes a privileged app's **public interface**) and **implicit** (two apps sharing the same **signing key** via **shareUserId** share the same UID and permissions).
- **URL-Sourced Malware / Banking Trojans** — **Zeus/SpyEye** first infect the desktop browser (inject a frame, steal base credentials), then show a fake security screen prompting an **APK download**; once installed, the Trojan **intercepts all incoming SMS** and forwards them, **silently bypassing SMS-based 2FA (OTP theft)**. *Countermeasure:* disable **Unknown Sources** (some carriers disable it by default, re-enableable only with root).
- **Pre-installed risks** — **Carrier IQ** (a carrier-installed rootkit transmitting IMEI/GPS/keystrokes); **HTC Logger** (`htcloggers.apk`) used `logcat` to collect locations/emails/keystrokes/SMS and **opened a local port any app with **INTERNET** could read**; **Google Wallet** stores card data in a hardware **Secure Element (SE)** behind a 4-digit PIN, but the **hashed PIN is NOT stored in the SE**, enabling **offline brute-force** (so rooting compromises Wallet; Google offers 100% Purchase Protection within 180 days).
- **Android as a hacking platform** — **Shark for Root**, cross-compiled `tcpdump`, **Network Spoofer** (ARP spoofing), **Connect Cat** (`netcat`),

Nmap. *Defenses:* physical security, strong PIN/password, no unknown-source apps, AV, FDE, constant OS updates (possibly via custom ROM).

11.8 iOS History

iOS descends from Steve Jobs's **NeXT** and **NeXTSTEP** (built on CMU's **Mach** kernel + **BSD Unix**, **Objective-C**). Apple bought NeXT in 1996 (→ **OPENSTEP** → **Mac OS X** in 2001). The iPhone (2007) ran "iPhone OS," later **iOS**; being a Mac OS X derivative, iOS uses the **Mach/BSD kernel**, **Objective-C**, and historically ran on **32-bit ARMv6/ARMv7**.

11.9 iOS Security Landscape

Early iOS was severely insecure: all apps ran as root, with no sandboxing, no code signing, no ASLR, no PIE. Later versions added: third-party apps confined to a low-privilege "mobile" account inside a **sandbox**; **mandatory code signing** (verified at load and runtime); **ASLR**; and **PIE** (apps load at a different base address each run). The **iPhone 3GS** introduced **hardware-accelerated AES** by default — the key is in flash but locked with the user's **PIN** — plus an optional **wipe after 10 failed PIN attempts**. Even so, targeted programs like the NSA's "**DROPOUT JEEP**" exist.

11.10 Jailbreaking iOS

Jailbreaking takes full **root** control, **permanently disabling code-signature verification** to install unapproved software (deep UI customisation, SSH/VNC, native compilation). The hub is **Cydia** (Jay Freeman / "saurik"); **Trollstore 2** now allows indefinite sideloading without a jailbreak (removing the 7-day resign limit). Risks: disabling code signing exposes many vectors, and vendors can **detect jailbroken status and block apps** (e.g. Apple with iBooks).

- **Boot-based jailbreak:** download the firmware **IPSW**, use exploit software (**redsn0w**, **greenpois0n**, **limera1n**, or the modern **checkm8/checkra1n** bootrom family), connect via USB, and enter **DFU mode** (hold **Power+Home 10s**, then release Power holding Home 5–10s).
- **Remote jailbreak:** **Jailbreakme.com** — loading a crafted **PDF** exploits **MobileSafari** (Jailbreakme 3.0 chained a **PDF-rendering bug** + a **kernel bug**), jailbreaking silently from the browser with no computer.

11.11 Hacking Other iPhones

An **un-jailbroken** iPhone is very safe remotely — a tiny attack surface (**one open port, TCP 62087**, no SSH/SMB/HTTP). Historic remote bugs: **CVE-2009-1683** (an ICMP request causing a device reset) and **CVE-2009-2204** (SMS arbitrary-code execution, demonstrated by **Charlie Miller**). Realistic vectors need physical/local/client-side access:

- **Physical:** a boot-based jailbreak silently installs SSH and reads secure data (including the **Keychain**) in **~6 minutes**.
- **Wireless MITM — the 2011 DEF CON "FOCUS 11":** attackers built a rogue AP (**Evil Twin**) on a Mac, exploited a **certificate-chain validation** flaw for a perfect **SSL MITM**, then served the **Jailbreakme 3.0 PDF** via MobileSafari to root the phone and install **SSH + VNC**.
- **Client-side:** targets apps like MobileSafari — but exploiting one app only yields its **sandbox data**; **escaping the sandbox to root requires a much harder kernel-level exploit**.
- **Jailbroken phones** are distinctly vulnerable if default credentials aren't changed: the **2009 "iKee" worm** hit jailbroken iPhones running OpenSSH with the default **alpine** password (first Rickrolling wallpapers, later building **botnets**). *Defenses:* don't jailbreak, or at minimum change the default password, enable SSH only when needed (SBSettings), and patch. (*Note: updating iOS removes the jailbreak, and jailbreaking requires staying on vulnerable firmware.*)
- **Malicious/vulnerable apps:** some hid code past Apple review — "**Handy Light**" (flashlight hiding tethering), "**InstaStock**" (Charlie Miller, hiding unsigned code). Legitimate apps shipped flaws: an old **Citi Mobile** app stored raw banking data insecurely, a **PayPal** app lacked **X.509 certificate validation** (MITM), and **Skype** had an **XSS** in the FullName field.

11.12 General Mobile Defense Summary

Maintain strict **physical control**; use screen locks (**PIN > 6 digits**) and keep the screen clean against **smudge attacks**; set Wi-Fi to "**Ask to Join**"; leave devices home during international travel; avoid storing sensitive data natively; use trusted third-party **encryption** (e.g. McAfee); centralise via **MDM** with **remote wipe/tracking**; update the OS constantly; and install apps only from trustworthy vendors.

End of guide — this document now covers Chapters 1–4 and 6–11 of the course. Chapters 5 (Unix Hacking) and 10 (Web Hacking) were marked ==REMOVED== in the source theory document; if you obtain their text, they can be slotted in to complete the set.